



**Masarykovo gymnázium,
SZŠ a VOŠ zdravotnická Vsetín**

Thomas Muijsenberg

8.AV

Platební systém

Maturitní práce

Vedoucí práce: Mgr. Vladislav Válek

2026

Prohlašuji, že jsem maturitní práci vypracoval samostatně s využitím uvedených pramenů a literatury.

Ve Vsetíně dne 1. dubna 2026

.....

(podpis autora práce)

Souhlas se zveřejněním

Souhlasím se zveřejněním své maturitní práce pro potřeby školy.

Ve Vsetíně dne 1. dubna 2026

.....

(podpis autora práce)

Poděkování

Na tomto místě bych rád poděkoval Mgr. Vladislavu Válkovi za odborné vedení, pomoc a rady během zpracování této maturitní práce. Jeho podněty mi pomohly lépe uchopit zvolené téma i jednotlivé souvislosti.

ANOTACE

Tato maturitní práce se zabývá návrhem a implementací bezhotovostního platebního systému určeného pro akce, jako jsou festivaly, školní jarmarky či trhy. Systém umožňuje účastníkům akce nabít si předplacený kredit na RFID/NFC kartu u pokladního stánku a následně bezhotovostně platit u různých prodejních stánků bez nutnosti manipulace s hotovostí. Aplikace je realizována jako webová aplikace s backendem ve frameworku Flask (Python) a databází PostgreSQL, frontend využívá vanilla JavaScript s komunikací s USB čtečkami karet prostřednictvím Web Serial API. Systém implementuje správu akcí, stánků, produktů, kategorií a zaměstnanců s rolemi (administrátor, manažer, pokladní, prodejce). Důraz je kladen na bezpečnost a integritu finančních dat – transakce jsou atomické a neměnné, aplikace využívá zamykání na úrovni řádků, idempotentní API, hashování hesel algoritmem Argon2 a parametrizované SQL dotazy. Součástí systému je také historie transakcí, statistiky prodejů, funkce undo/redo a možnost klonování akcí.

ANNOTATION

This graduation thesis deals with the design and implementation of a cashless payment system intended for events such as festivals, school fairs, or markets. The system allows event attendees to load prepaid credit onto an RFID/NFC card at a cashier booth and then pay contactlessly at various seller booths without the need to handle cash. The application is implemented as a web application with a backend built on the Flask framework (Python) and a PostgreSQL database; the frontend uses vanilla JavaScript with communication to USB card readers via the Web Serial API. The system implements management of events, booths, products, categories, and employees with roles (administrator, manager, cashier, seller). Emphasis is placed on the security and integrity of financial data – transactions are atomic and immutable, the application utilizes row-level locking, an idempotent API, Argon2 password hashing, and parameterized SQL queries. The system also includes transaction history, sales statistics, undo/redo functionality, and the ability to clone events.

Obsah

ÚVOD	8
1 Historie a kontext	9
1.1 Vývoj bezhotovostních platebních systémů	9
1.2 Webové technologie a jejich role	9
1.3 RFID a NFC technologie	10
2 Teoretická východiska	10
2.1 Architektura webových aplikací	10
2.2 REST API a komunikace klient-server	11
2.3 Relační databáze a transakční zpracování	12
2.4 Soft-delete vzor	12
2.5 Idempotence v síťové komunikaci	13
2.6 Zamykání a souběžný přístup	13
3 Použité technologie a nástroje	14
3.1 Python	14
3.2 Flask	15
3.3 PostgreSQL	15
3.4 JavaScript a Web Serial API	16
3.5 Další technologie a knihovny	17
4 Bezpečnostní aspekty	17
4.1 Autentizace a správa hesel	17
4.2 Správa sessions	18
4.3 Ochrana proti běžným útokům	18
4.4 Zabezpečení finančních operací	19
5 Shrnutí teoretické části a přechod k praktické části	19
PRAKTICKÁ ČÁST	21
6 Cíle práce a zadání	21

6.1 Zadání	21
6.2 Hlavní cíl	21
6.3 Dílčí cíle	21
7 Architektura systému	22
8 Prerekvizity a instalace	22
8.1 Systémové požadavky.....	22
8.2 Instalace a spuštění.....	23
8.3 Konfigurace	25
8.3.1 Přehled konfiguračních položek	25
9 Databáze a její rozvržení	31
9.1 Schéma databáze	31
9.2 Triggery a integritní omezení	33
9.3 Indexy.....	33
10 Způsob programování a využití nástrojů	34
10.1 Struktura projektu a Flask blueprinty	34
10.2 Připojení k databázi — psycopg a connection pool.....	36
10.3 Serverové sessions v PostgreSQL	38
10.4 Zpracování chyb — vlastní výjimky.....	38
10.5 Generátor SQL dotazů (Query Builder).....	39
10.6 Synchronizace vazebních tabulek	40
11 Práva přístupu	40
11.1 Hierarchie rolí	40
11.2 Vynucování oprávnění	41
11.3 Vzájemná výlučnost rolí.....	42
12 Čtení karet pomocí čteček	42
12.1 Komunikace přes Web Serial API.....	42
13 ACID operace a práce s databází při finančních operacích	44

13.1 Průběh platby	44
13.2 Idempotence a fingerprint	45
13.3 Refundace	45
13.4 Zajištění integrity na úrovni databáze	46
14 Caching	46
14.1 Serverový caching statických souborů	46
14.2 Klientský caching dat	47
14.3 Persistence stavu objednávky	49
15 Statistiky a historie plateb	49
15.1 Statistiky akcí	49
15.2 Historie transakcí	50
16 Kopírování (paste) a zpět/znovu (undo/redo)	51
16.1 Kopírování (paste)	51
16.2 Vrátit zpět a provést znovu (undo/redo)	52
17 Využití — jak vypadá používání aplikace	54
17.1 Přihlášení	54
17.2 Výběr akce a stánku	54
17.3 Pokladní rozhraní (index)	55
17.4 Správa akcí (event manager)	56
17.5 Správa zaměstnanců (admin)	57
17.6 Správa smazaných záznamů	58
17.7 Typický scénář obsluhy	58
17.7.1 Scénář pokladního (cashier):	58
17.7.2 Scénář prodejce (seller):	59
18 Bezpečnostní implementace	60
18.1 Hashování hesel	60
18.2 Rate limiting	61

18.3 Validace nahrávaných souborů.....	61
18.4 Ochrana cookies a sessions	61
18.5 Threat model — proti čemu se systém brání	62
18.6 Validace vstupních dat	62
18.6.1 Co je mimo scope systému	64
19 Plánované úlohy na pozadí a logování.....	65
19.1 Plánované úlohy.....	65
19.1.1 Zálohování databáze.....	65
19.2 Logování	66
19.2.1 Logování transakcí.....	67
20 Frontend a uživatelské rozhraní	67
20.1 Architektura frontendu	67
20.2 Event delegation.....	67
20.3 Vyhledávání a řazení tabulek	68
20.3.1 Vyhledávání	68
20.3.2 Řazení	69
21 Testování	70
21.1 Unit testy a integrační testy	71
21.2 Kritické scénáře — platby a refundace	71
21.3 Testy undo/redo a paste	72
21.4 Další testované oblasti	72
ZÁVĚR.....	73
SEZNAM POUŽITÉ LITERATURY	76
SEZNAM PŘÍLOH.....	77
Příloha 1 – Seznam pro vybírání předčísí	78
Příloha 2 – Platební operace – flowchart.....	79

ÚVOD

Bezhotovostní platební systémy se v posledních letech staly nedílnou součástí organizace kulturních, sportovních a společenských akcí. Tradiční hotovostní platby přinášejí řadu praktických obtíží — od nutnosti manipulovat s mincemi a bankovkami přes riziko krádeží až po zdlouhavé účtování a nemožnost sledovat prodeje v reálném čase. Moderní technologie nabízejí elegantní alternativu v podobě systémů založených na bezkontaktních kartách, které umožňují rychlé, přehledné a bezpečné transakce.

Tato práce se zabývá návrhem a realizací webové aplikace sloužící jako bezhotovostní platební systém určený primárně pro festivaly, školní akce, trhy a podobné události. Systém využívá RFID/NFC karty a čtečky k identifikaci uživatelů a jejich virtuálních peněženek. Princip fungování je jednoduchý: návštěvník akce si na pokladním stánku nabije finanční prostředky na kartu a následně může platit u libovolného prodejního stánku pouhým přiložením karty ke čtečce. Odpadá tak potřeba manipulace s hotovostí, zrychluje se obsluha a organizátoři získávají kompletní přehled o všech transakcích.

1 Historie a kontext

1.1 Vývoj bezhotovostních platebních systémů

Historie bezhotovostního placení sahá až do 50. let 20. století, kdy se začaly používat první platební karty. Skutečný rozmach však přišel s rozvojem elektronických terminálů v 80. a 90. letech. Na přelomu tisíciletí se objevily první bezkontaktní technologie založené na standardech RFID (Radio-Frequency Identification) a později NFC (Near Field Communication), které umožnily platbu pouhým přiložením karty nebo telefonu k terminálu.

V prostředí festivalů a uzavřených akcí se tyto technologie začaly prosazovat přibližně od roku 2010. Systémy jako Intellipay, Tappit nebo český Albi Pay umožňují organizátorům vytvořit uzavřený platební ekosystém, ve kterém návštěvníci platí pomocí náramků nebo karet s RFID/NFC čipem. Hlavní výhody těchto systémů spočívají ve zrychlení obsluhy, eliminaci hotovosti, snadnějším účtování a možnosti sledovat prodeje v reálném čase.

1.2 Webové technologie a jejich role

Současný vývoj webových aplikací prošel za posledních dvacet let dramatickou transformací. Od statických HTML stránek se posunul k dynamickým aplikacím schopným soupeřit s desktopovým softwarem. Klíčovou roli v tomto vývoji sehrál jazyk JavaScript na straně klienta a vznik serverových frameworků v jazycích jako Python, Ruby, Java nebo JavaScript (Node.js).

Python se stal jedním z nejpobulárnějších programovacích jazyků díky své čitelnosti, rozsáhlému ekosystému knihoven a silné komunitě. V oblasti webového vývoje nabízí Python několik frameworků — od minimalistického Flasku přes robustní Django až po moderní FastAPI. Pro projekt bezhotovostního platebního systému byl zvolen Flask, jehož modulární architektura a nízká bariéra vstupu umožňují rychlý vývoj a snadnou údržbu.

Na straně databázových systémů dominuje PostgreSQL jako jeden z nejvyspělejších open-source relačních databázových systémů. PostgreSQL nabízí pokročilé funkce jako JSONB datový typ, trigger, advisory zámky a robustní transakční mechanismus, které jsou pro finanční systém klíčové.

1.3 RFID a NFC technologie

RFID (Radio-Frequency Identification) je technologie umožňující bezdrátovou identifikaci objektů pomocí rádiových vln. RFID systém se skládá ze dvou základních komponent: tagu (čipu) umístěného na identifikovaném objektu a čtečky, která vysílá rádiový signál a přijímá odpověď z tagu. Pasivní RFID tagy nemají vlastní zdroj energie — jsou napájeny elektromagnetickým polem čtečky, což umožňuje jejich miniaturizaci a nízkou cenu.

NFC (Near Field Communication) je podmnožina RFID. NFC Forum popisuje NFC jako krátkodosahovou technologii na frekvenci 13,56 MHz, s komunikací mezi zařízeními vzdálenými jen několik centimetrů, což ji činí vhodnou pro bezpečné bezkontaktní platby díky minimalizaci rizika nechtěného načtení. NFC přináší oproti obecnému RFID výhodu ve standardizovaném komunikačním protokolu a široké podpoře v mobilních zařízeních.

V kontextu tohoto projektu slouží RFID/NFC karty jako nosič jednoznačného identifikátoru (tag ID), který je při přiložení ke čtečce přenesen do webové aplikace. Čtečka je připojena k počítači přes USB a komunikuje prostřednictvím sériového portu. Webová aplikace využívá Web Serial API prohlížeče pro přímou komunikaci se čtečkou bez potřeby dalšího softwaru.

2 Teoretická východiska

2.1 Architektura webových aplikací

Webové aplikace typicky fungují na principu klient-server architektury. Klient (webový prohlížeč) odesílá HTTP požadavky na server, který je zpracuje a vrátí odpověď — nejčastěji ve formátu HTML, JSON nebo jiných datových formátů. Tato

architektura přináší výhodu centralizované správy dat a logiky na serveru, zatímco klient se stará o prezentaci a uživatelskou interakci.

V moderních webových aplikacích se ustálily dva hlavní přístupy: tradiční serverové vykreslování (SSR — Server-Side Rendering), kde server generuje kompletní HTML stránky, a jednostránkové aplikace (SPA — Single Page Application), kde se počáteční stránka načte jednou a veškerá další komunikace se serverem probíhá pomocí asynchronních požadavků (AJAX/fetch), přičemž se aktualizují pouze části stránky.

Tento projekt využívá hybridní přístup — server vykresluje základní HTML šablony pomocí šablonovacího systému Jinja2, ale veškerý dynamický obsah je načítán a aktualizován prostřednictvím JavaScriptového kódu, který komunikuje s REST API endpointy serveru. Tento přístup kombinuje výhody obou metod: rychlé počáteční načtení stránky a plynulou interakci bez nutnosti znovunačítání.

2.2 REST API a komunikace klient-server

Následující definice REST API byla vygenerována pomocí Claude Opus 4.6 (Anthropic, 2026):

„REST (Representational State Transfer) je architektonický styl pro návrh síťových aplikací. REST API definuje sadu konvencí pro komunikaci mezi klientem a serverem pomocí standardních HTTP metod:

- **GET** — získání dat (čtení),
- **POST** — vytvoření nového záznamu nebo provedení akce,
- **PUT/PATCH** — aktualizace existujícího záznamu,
- **DELETE** — smazání záznamu.

Každý endpoint API představuje určitý zdroj (resource) identifikovaný URL adresou. Server vrací data typicky ve formátu JSON, který je nativně podporován JavaScriptem a snadno zpracovatelný na obou stranách komunikace.“

V tomto projektu jsou API endpointy organizovány do logických skupin (blueprintů) podle domény — transakce, uživatelé, události, produkty a další. Klient využívá

nativní funkci `fetch()` pro komunikaci se serverem, což eliminuje závislost na externích knihovnách.

2.3 Relační databáze a transakční zpracování

Relační databáze ukládají data ve formě tabulek propojených vztahy (relacemi). Tento model, poprvé formalizovaný Edgarem F. Coddem v roce 1970, se stal standardem pro strukturovaná data díky své matematické podloženosti, flexibilitě dotazovacího jazyka SQL a silným záručním mechanismům.

Pro finanční systémy je naprosto zásadní koncept ACID transakcí:

- **Atomicita** (Atomicity) — transakce je buď provedena celá, nebo vůbec. Pokud jakákoli část operace selže, všechny změny jsou vráceny zpět.
- **Konzistence** (Consistency) — transakce převede databázi z jednoho platného stavu do jiného platného stavu. Všechna omezení (constraints) musí být splněna.
- **Izolace** (Isolation) — souběžně probíhající transakce se navzájem neovlivňují. Každá transakce vidí konzistentní stav dat.
- **Trvanlivost** (Durability) — jakmile je transakce potvrzena (committed), její změny jsou trvale uloženy i v případě výpadku systému.

V kontextu platebního systému znamená dodržení ACID vlastností například to, že platba je buď kompletně provedena (odečtena z peněženky a zaznamenána jako transakce), nebo k ní vůbec nedojde — nikdy nemůže nastat situace, kdy by se peníze odečetly, ale transakce by nebyla zaznamenána, či naopak.

2.4 Soft-delete vzor

Soft-delete je návrhový vzor, při kterém se záznamy z databáze fyzicky neodstraňují, ale pouze se označí jako smazané — typicky nastavením časového razítka do sloupce `deleted_at`. Tento přístup přináší několik výhod:

- **Obnovitelnost** — smazané záznamy lze kdykoliv obnovit nastavením `deleted_at` zpět na `NULL`.

- **Auditní stopa** — je zachována kompletní historie všech dat, což je u finančního systému obzvláště důležité.
- **Referenční integrita** — záznamy, na které odkazují jiné tabulky (například transakce odkazující na smazaného uživatele), zůstávají v databázi a nedojde k porušení cizích klíčů.
- **Bezpečnost** — eliminuje se riziko nechtěného nevratného smazání dat.

Nevýhodou soft-delete je nutnost přidávat podmínku `WHERE deleted_at IS NULL` do většiny dotazů a potenciální nárůst velikosti databáze. Tyto nevýhody jsou však v kontextu finančního systému zanedbatelné ve srovnání s přínosy.

2.5 Idempotence v síťové komunikaci

Idempotence je vlastnost operace, která zaručuje, že opakované provedení stejné operace má stejný výsledek jako její jednorázové provedení. V kontextu webových API je idempotence kriticky důležitá, protože síťová komunikace je ze své podstaty nespolehlivá — požadavek může být odeslán vícekrát kvůli výpadku spojení, opakovanému kliknutí uživatele nebo chybě na straně klienta.

U finančních transakcí má nedodržení idempotence potenciálně katastrofální důsledky — dvojité provedení platby znamená dvojité odečtení peněz z účtu. Řešením je zavedení idempotentního klíče (idempotency key), který jednoznačně identifikuje každou zamýšlenou operaci. Server si zapamatuje výsledek operace spojeného s daným klíčem a při opakovaném požadavku se stejným klíčem vrátí uložený výsledek namísto opětovného provedení operace.

2.6 Zamykání a souběžný přístup

Souběžný přístup více uživatelů k databázi vyžaduje mechanismy, které zabrání vzájemnému narušení dat. Existuje několik úrovní zamykání:

- **Řádkové zámky** (row-level locks) — zamykají konkrétní řádky v tabulce. Používají se typicky při aktualizaci konkrétního záznamu, například zůstatku peněženky.

- **Kooperativní zámky** (advisory locks) — jsou zámky na úrovni aplikace, které databáze spravuje, ale nevynucuje automaticky. Aplikace je používá pro koordinaci logických operací, které by se neměly provádět souběžně.

V platebním systému se řádkové zámky využívají při manipulaci se zůstatkem peněženky — klauzule `SELECT . . . FOR UPDATE` zajistí, že dva souběžné pokusy o platbu ze stejné peněženky nepovedou k nekonzistentnímu stavu. Kooperativní zámky slouží k zamezení souběžného provádění operací undo/redo nebo hromadného kopírování dat jedním zaměstnancem.

3 Použité technologie a nástroje

3.1 Python

Následující popis jazyka Python byl vygenerován pomocí Claude Opus 4.6 (Anthropic, 2026):

„Python je vysokoúrovňový, interpretovaný programovací jazyk s dynamickým typováním a automatickou správou paměti. Vyznačuje se čitelnou syntaxí, která klade důraz na srozumitelnost kódu, a rozsáhlou standardní knihovnou. Python podporuje více programovacích paradigmat — procedurální, objektově orientované i funkcionální programování.

Pro vývoj webových aplikací je Python vhodný díky široké nabídce frameworků, aktivní komunitě a velkému množství kvalitních knihoven pro práci s databázemi, zabezpečením, validací dat a dalšími úlohami. Alternativami pro vývoj webových aplikací by mohly být JavaScript/TypeScript s Node.js (výhoda sdílení jazyka mezi klientem a serverem), Java se Spring frameworkem (robustní, ale těžkopádnější), Go (vysoký výkon, ale menší ekosystém pro webový vývoj) nebo PHP s Larevelem (tradiční volba pro webové aplikace).“

Python s Flaskem byl zvolen pro svou jednoduchost, flexibilitu a silný ekosystém bezpečnostních knihoven.

3.2 Flask

Flask je mikroframework pro webové aplikace v Pythonu. Označení „mikro“ neznamena, že by byl omezený — naopak, Flask poskytuje jádro pro zpracování HTTP požadavků, směrování, šablonování a správu sessions a umožňuje rozšíření pomocí pluginů (extensions) podle potřeb konkrétního projektu.

Hlavní výhodou Flasku oproti robustnějším frameworkům jako Django je svoboda ve výběru komponent — vývojář není nucen používat konkrétní ORM, šablonovací systém nebo autentizační mechanismus. To je obzvláště výhodné pro projekt, kde je potřeba přímá kontrola nad databázovými dotazy (namísto ORM) a vlastní implementace session mechanismu.

Flask používá systém blueprintů pro modularizaci aplikace. Blueprint je logická skupina pohledů (views), šablon a statických souborů, která může být registrována do hlavní aplikace. Tento přístup umožňuje rozdělit velkou aplikaci do samostatných, snadno spravovatelných modulů.

3.3 PostgreSQL

PostgreSQL je pokročilý open-source relační databázový systém s více než 35letou historií vývoje. Oproti jiným databázím (MySQL, SQLite, MariaDB) vyniká pokročilými funkcemi:

- **JSONB** — dle dokumentace PostgreSQL se jedná o binární formát pro ukládání JSON dat s podporou indexování a dotazování. Umožňuje kombinovat výhody relačního a dokumentového modelu.
- **Triggery** — procedury automaticky spouštěné při určitých databázových událostech (INSERT, UPDATE, DELETE). Umožňují implementovat business logiku přímo na úrovni databáze.
- **Kooperativní zámky** (advisory locks) — aplikační zámky spravované databází pro koordinaci souběžných operací.

- **Pokročilé indexy**¹ — parciální indexy (pouze pro podmnožinu řádků), expression indexy (nad výrazem, například `LOWER (name)`), GIN indexy pro JSONB.
- **Silný transakční model** — plná podpora ACID s pokročilými úrovněmi izolace.

SQLite by pro tento projekt nebyl vhodný kvůli omezenému souběžnému přístupu a absenci pokročilých funkcí. MySQL by byl funkčně dostačující, ale PostgreSQL nabízí lepší podporu pro triggery, JSONB a kooperativní zámky, které jsou v projektu intenzivně využívány.

3.4 JavaScript a Web Serial API

Na straně klienta je použit čistý JavaScript (vanilla JS) bez použití frameworků jako React, Vue nebo Angular. Tento přístup eliminuje závislost na externích nástrojích pro sestavení (build tools) a zjednodušuje nasazení — stačí servírovat statické soubory bez nutnosti kompilace.

JavaScript je organizován do ES modulů (ECMAScript modules), což umožňuje logické rozdělení kódu do samostatných souborů s jasně definovanými importy a exporty. Moderní prohlížeče nativně podporují ES moduly prostřednictvím atributu `type="module"` ve skriptových tagech.

Web Serial API je relativně nové rozhraní prohlížeče umožňující přímou komunikaci se zařízeními připojenými přes sériový port (USB, Bluetooth). Toto API je klíčové pro čtení dat z RFID/NFC čteček bez nutnosti instalace dalšího softwaru nebo ovladačů. Dle MDN je API v současnosti podporováno v prohlížečích založených na Chromiu. (Google Chrome, Microsoft Edge, Opera).

¹ Index je pomocná datová struktura udržovaná databází vedle samotných dat, která funguje jako rejstřík v knize — umožňuje rychle vyhledat relevantní záznamy bez nutnosti procházet celou tabulku. Parciální index pokrývá pouze řádky splňující zadanou podmínku, čímž je menší a efektivnější.

3.5 Další technologie a knihovny

Projekt využívá řadu dalších technologií a knihoven:

- **psycopg 3** — moderní PostgreSQL adaptér pro Python s podporou connection pooling, parametrizovaných dotazů a asynchronního přístupu.
- **Argon2** — Dle doporučení OWASP Password Storage Cheat Sheet (2024) je Argon2id preferovanou volbou pro hashování² hesel. Algoritmus zvítězil v soutěži Password Hashing Competition (2015) a je považovaný za nejbezpečnější algoritmus pro hashování hesel. Je odolný vůči útokům hrubou silou i specializovanému hardwaru (GPU, ASIC).
- **Chart.js** — JavaScriptová knihovna pro vytváření interaktivních grafů. Použita pro vizualizaci statistik akcí.
- **Jinja2** — šablonovací engine pro Python, integrovaný do Flasku. Umožňuje generovat HTML s dynamickým obsahem.
- **APScheduler** — knihovna pro plánování periodických úloh na pozadí (například čištění expirovaných sessions).
- **Flask-Limiter** — rozšíření Flasku pro omezení počtu požadavků (rate limiting) jako ochrana proti DoS útokům.
- **Pillow** — knihovna pro zpracování a validaci obrázků.

4 Bezpečnostní aspekty

4.1 Autentizace a správa hesel

Autentizace je proces ověření identity uživatele. V webových aplikacích se nejčastěji provádí pomocí uživatelského jména (nebo e-mailu) a hesla. Kritickým aspektem je způsob ukládání hesel — hesla se nikdy neukládají v čitelné podobě, ale jako hash. Při ověření se hashuje zadané heslo a porovná se s uloženým hashem.

² Hashování je jednosměrná matematická transformace, která převede vstupní data (např. heslo) na řetězec pevné délky — z výsledného hashe nelze zpětně získat původní vstup, ale stejný vstup vždy vyprodukuje stejný výstup, což umožňuje ověření hesla bez jeho ukládání v čitelné podobě.

Moderní hashovací algoritmy pro hesla (Argon2, bcrypt, scrypt) jsou záměrně pomalé a paměťově náročné, aby ztížily útok hrubou silou. Argon2, použitý v tomto projektu, má konfigurovatelné parametry pro časovou náročnost (time cost), paměťovou náročnost (memory cost) a míru paralelismu, což umožňuje přizpůsobit bezpečnostní úroveň dostupnému hardwaru.

4.2 Správa sessions

Po úspěšné autentizaci je nutné udržovat informaci o přihlášení uživatele napříč požadavky. K tomu slouží sessions — mechanismus pro udržování stavu v jinak bezstavovém protokolu HTTP. Existují dva základní přístupy:

- **Klientské sessions** — veškerá data jsou uložena v cookie na straně klienta (typicky šifrovaná a podepsaná). Výhodou je jednoduchost, nevýhodou omezená velikost a nemožnost serverové invalidace.
- **Serverové sessions** — v cookie je uložen pouze identifikátor session (session ID) a vlastní data jsou uložena na serveru (v databázi, Redis, souborovém systému). Výhodou je plná kontrola nad životním cyklem session a možnost invalidace.

Pro finanční systém je serverové ukládání sessions bezpečnější, protože umožňuje okamžitou invalidaci session (například při odhlášení nebo podezření na kompromitaci) a neumožňuje klientovi manipulovat se session daty.

4.3 Ochrana proti běžným útokům

Webové aplikace čelí řadě bezpečnostních hrozeb. Mezi nejzávažnější patří:

- **SQL injection** — útočník vloží škodlivý SQL kód do vstupních polí. Ochranou je důsledné používání parametrizovaných dotazů, kdy jsou uživatelská data oddělena od SQL příkazů.
- **Cross-Site Scripting (XSS)** — útočník vloží škodlivý JavaScript kód, který se spustí v prohlížeči jiného uživatele. Ochranou je escapování uživatelských dat při vkládání do HTML.

- **Cross-Site Request Forgery (CSRF)** — útočník přiměje přihlášeného uživatele k nevědomému provedení akce. Ochranou je nastavení cookie atributu SameSite a případně použití CSRF tokenů.
- **Brute-force útoky** — systematické zkoušení hesel. Ochranou je omezení počtu pokusů o přihlášení (rate limiting).

4.4 Zabezpečení finančních operací

Finanční operace vyžadují zvláštní pozornost z hlediska bezpečnosti. Klíčové aspekty zahrnují:

- **Neměnnost záznamů** — transakce jednou zapsané do systému nesmí být měnitelné ani smazatelné, aby byla zajištěna auditovatelnost a integrita finančních dat.
- **Kontrola oprávnění** — každá finanční operace musí být autorizována na více úrovních (aplikační logika i databázové trigger).
- **Atomicita operací** — platba musí být provedena kompletně (odečtení z peněženky + zápis transakce), nebo vůbec.
- **Ochrana proti duplicitním operacím** — mechanismus idempotence zabraňuje dvojitému provedení téže platby.

5 Shrnutí teoretické části a přechod k praktické části

V teoretické části byly představeny principy a technologie, na kterých je platební systém postaven. Byly popsány koncepty webové architektury klient-server, REST API, relačních databází s ACID transakcemi, bezpečnostního zabezpečení webových aplikací a komunikace s hardwarovými zařízeními prostřednictvím Web Serial API. Dále byly představeny konkrétní technologie — Python s Flaskem, PostgreSQL, JavaScript a další knihovny — společně se zdůvodněním jejich výběru.

V následující praktické části bude podrobně popsána samotná implementace systému: struktura projektu, databázové schéma, způsob komunikace s kartovými čtečkami, systém oprávnění, finanční operace, caching, statistiky a další funkce.

Praktická část obsahuje ukázky kódu, diagramy a konkrétní popisy řešení jednotlivých problémů.

PRAKTICKÁ ČÁST

6 Cíle práce a zadání

6.1 Zadání

Zadání maturitní práce zní: „Cashier Systém — Kompletní systém pro bezhotovostní platby na hromadných akcích, součástí bude i pokladna pro nabíjení čipů, databázový systém, jednotlivé terminálové kasy, statistiky, výpisy, přehledy. Různé role a práva přístupu, kontrolní mechanismy, zálohování, offline status.“

6.2 Hlavní cíl

Navrhnout a implementovat funkční, bezpečný a snadno nasaditelný bezhotovostní platební systém pro hromadné akce, který bude použitelný prostřednictvím webového prohlížeče bez nutnosti instalace speciálního softwaru.

6.3 Dílčí cíle

1. **Správa více souběžných akcí** — systém umožní organizátorům vytvářet a spravovat více akcí současně, každou s vlastní sadou stánků, produktů, kategorií a zaměstnanců.
2. **Integrita a bezpečnost finančních dat** — finanční transakce budou atomické a neměnné, systém zajistí konzistenci dat i při souběžném přístupu více uživatelů a ochrání se proti běžným bezpečnostním hrozbám (SQL injection, duplicitní transakce, neoprávněný přístup).
3. **Statistiky a přehledy** — systém poskytne organizátorům detailní statistiky prodeje a historii transakcí na úrovni akce, stánků i jednotlivých produktů.

7 Architektura systému

Aplikace využívá třívrstvou architekturu klient-server:

- **Prezentační vrstva (klient)** — webový prohlížeč s JavaScriptovým kódem, který komunikuje se serverem přes REST API a se čtečkou karet přes Web Serial API.
- **Aplikační vrstva (server)** — Flask aplikace v Pythonu, která zpracovává HTTP požadavky, provádí autentizaci a autorizaci, validuje vstupy a komunikuje s databází.
- **Datová vrstva (databáze)** — PostgreSQL databáze s 17 tabulkami, rozsáhlou soustavou triggerů a integritních omezení. Významná část business logiky (validace transakcí, ochrana neměnnosti, normalizace dat) je implementována přímo na úrovni databáze.

Komunikace mezi vrstvami probíhá následovně: uživatel (zaměstnanec) interaguje s webovým rozhraním v prohlížeči. JavaScript odesílá požadavky na Flask server, který je zpracuje a provede odpovídající databázové operace. Výsledky se vrací ve formátu JSON a JavaScript aktualizuje zobrazení. Čtečka RFID/NFC karet je připojena přímo k prohlížeči prostřednictvím Web Serial API — server se o komunikaci se čtečkou nestará, pouze přijímá tag ID karty jako parametr požadavku.

8 Prerekvizity a instalace

8.1 Systémové požadavky

Pro spuštění aplikace je potřeba:

- **Python 3.10+** — aplikace využívá moderní syntaxi (match/case, type hints s `|`).
- **PostgreSQL 14+** — databázový server s podporou rozšíření `pgcrypto`.
- **Webový prohlížeč s podporou Web Serial API** — Google Chrome, Microsoft Edge nebo Opera (pro čtení karet).

- **Git** — systém správy verzí pro klonování repozitáře a správu zdrojového kódu (potřebný pouze pro instalaci).
- **RFID/NFC čtečka** — připojená přes USB, komunikující přes **sériový port** (pokud čtečka zároveň komunikuje jinými způsoby, bude je nejspíš nutné vypnout).
- **Node.js 18+** — potřebný pro spuštění testů.

8.2 Instalace a spuštění

1. Získání projektu:

```
git clone https://github.com/mgvsetin/2426-mp-ThomasMuij.git
cd 2426-mp-ThomasMuij
```

Případně stáhněte ZIP archiv projektu a rozbalte ho.

2. Vytvoření virtuálního prostředí (doporučeno):

```
python -m venv .venv
```

Aktivace (windows):

```
.venv\Scripts\activate
```

Aktivace (Linux/macOS):

```
source .venv/bin/activate
```

3. Instalace Python závislostí:

```
pip install -r requirements.txt
```

Hlavní závislosti zahrnují: Flask, pycopg (s poolem), argon2-cffi, Flask-Limiter, APScheduler, Pillow, phonenumbers, email-validator a python-dateutil.

4. Příprava databáze:

Vytvořte PostgreSQL databázi a uživatele (přes psql nebo pgAdmin):


```
CREATE DATABASE cashier_app;
```

Pro testování:

```
CREATE DATABASE cashier_app_test;
```

Na systémech Debian/Ubuntu může být nutné doinstalovat balík postgresql-contrib pro rozšíření pgcrypto (rozšíření se aktivuje automaticky při inicializaci schématu).

Nastavte připojovací údaje buď proměnnou prostředí `DATABASE_CONNINFO`, nebo v souboru `instance/config.py` (nezapomeňte změnit heslo):

```
DATABASE_CONNINFO = "dbname=cashier_app host=localhost user=postgres  
password=heslo port=5432"
```

5. Inicializace databázového schématu:

```
flask --app cashier_app init-db
```

6. Vytvoření prvního administrátora:

```
flask --app cashier_app create-admin
```

Příkaz interaktivně vyžádá uživatelské jméno, e-mail a heslo (s potvrzením). Vstup je opakovaně vyžádán, dokud není zadána platná hodnota. Heslo je okamžitě zahashováno algoritmem Argon2. Vytvořený zaměstnanec má `is_admin=TRUE` a slouží jako první správce systému. Příkaz lze také použít při zapomenutí hesla všech administrátorů.

7. Pro testování lze vložit testovací data:

```
flask --app cashier_app insert-development-values
```

8. Spuštění vývojového serveru:

```
flask --app cashier_app run
```

Aplikace bude dostupná na `http://localhost:5000`.

9. Produkční nasazení:

Pro produkci se doporučuje použít WSGI server (například Gunicorn) za reverzním proxy serverem (nginx). Nginx by měl obsluhovat statické soubory a složku s nahranými obrázky produktů. V konfiguraci je nutné:

nastavit `SESSION_COOKIE_SECURE = True`,

vygenerovat silný `SECRET_KEY` (např. `python -c "import secrets; print(secrets.token_urlsafe(32))"`),

nakonfigurovat `PROXY_FIX` podle nastavení reverzního proxy.

8.3 Konfigurace

Aplikace se konfiguruje prostřednictvím výchozích hodnot v tovární funkci `create_app()`, které lze přepsat souborem `instance/config.py` nebo proměnnými prostředí. Soubor `instance/config.py` se vytvoří automaticky při prvním spuštění aplikace (pokud neexistuje). Jedná se o běžný Python soubor — hodnoty se zapisují jako přiřazení do proměnných (např. `SECRET_KEY = "..."`). Hodnoty zapsané v tomto souboru přepíší výchozí nastavení z `create_app()`. Dvě hodnoty (`SECRET_KEY` a `DATABASE_CONNINFO`) lze alternativně nastavit proměnnými prostředí (`CASHIER_APP_SECRET` a `DATABASE_CONNINFO`) — proměnná prostředí má přednost před výchozí hodnotou, ale soubor `config.py` má přednost před oběma.

8.3.1 Přehled konfiguračních položek

Bezpečnost a sessions:

- **SECRET_KEY** — tajný klíč pro podepisování sessions. Výchozí hodnota je pouze ukázková (pro development) a nesmí se používat v produkci.

Doporučení: vygenerovat příkazem `python -c "import secrets; print(secrets.token_urlsafe(32))"` a vložit do `config.py`.

- **SESSION_COOKIE_SECURE** — určuje, zda se session cookie odesílá pouze přes HTTPS. Výchozí: False. Doporučení pro produkci: True (vyžaduje HTTPS).
- **SESSION_COOKIE_HTTPONLY** — zabraňuje přístupu JavaScriptu k session cookie. Výchozí: True. Doporučení: ponechat True.
- **SESSION_COOKIE_SAMESITE** — omezuje odesílání cookie při cross-site požadavcích. Výchozí: 'Lax'. Doporučení: 'Lax' nebo 'Strict'. Při nastavení na None je nutné přidat CSRF ochranu.
- **SESSION_ENFORCE_UA** — vynucuje kontrolu User-Agent při ověřování sessions. Výchozí: False. Doporučení pro produkci: False (User-Agent se často mění a může způsobovat falešné odhlášení).
- **SESSION_ENFORCE_IP** — vynucuje kontrolu IP adresy při ověřování sessions. Výchozí: False. Doporučení pro produkci: False (IP adresa se může měnit kvůli mobilním sítím, VPN nebo proxy).
- **SESSION_MAX_INACTIVE_DAYS** — počet dnů neaktivity, po kterém session vyprší. Výchozí: 7.

Databáze:

- **DATABASE_CONNINFO** — připojovací řetězec pro PostgreSQL ve formátu libpq. Výchozí: lokální připojení s heslem heslo123. Doporučení: nastavit skutečné přístupové údaje a silné heslo. Příklad: `"dbname=cashier_app host=localhost user=postgres password=silne_heslo port=5432"`.

Reverzní proxy:

- **PROXY_FIX** — slovník pro konfiguraci Werkzeug ProxyFix middleware. Výchozí: všechny hodnoty 0 (proxy není aktivní). Nastavuje se pouze pokud aplikace běží za reverzním proxy serverem (např. nginx). Klíče: `x_for` (počet důvěryhodných proxy pro X-Forwarded-For), `x_proto`, `x_host`, `x_port`,

`x_prefix` a volitelně `trusted_hosts` (seznam IP adres proxy). Pro produkci nastavte hodnoty podle nastavení reverzní proxy.

Čtečka karet:

- **READER_INFO** — slovník s nastavením sériového portu pro NFC/RFID čtečku. Povinný klíč `serial_port_options` obsahuje parametry: `baudRate` (výchozí: 9600), `dataBits` (8), `stopBits` (1), `parity` ('none'), `flowControl` ('none'). Volitelný klíč `filters` obsahuje seznam slovníků s USB vendor/product ID pro filtrování dostupných zařízení v prohlížeči. Doporučení: výchozí hodnoty jsou vhodné pro běžné čtečky (pokud má čtečka jiné, tak je nutné výchozí hodnoty změnit); filtry nastavit podle konkrétního modelu čtečky.

Undo a refundace:

- **MAX_UNDO_CHANGES** — maximální počet kroků zpět (undo) pro zaměstnance. Výchozí: 30. Doporučení: ponechat výchozí hodnotu.
- **UNDO_TIME_LIMIT_MINUTES** — časový limit v minutách, po jehož uplynutí již nelze provést undo. Výchozí: 60.
- **REFUND_TIME_LIMIT_MINUTES** — časový limit v minutách pro vrácení platby (refund). Výchozí: 5. Doporučení: nastavit podle provozních požadavků (např. 5–15 minut).

Nahrávání obrázků:

- **UPLOAD_FOLDER** — cesta k adresáři pro ukládání obrázků produktů. Výchozí: `instance/uploads/products/`. Doporučení: ponechat výchozí, případně nastavit absolutní cestu při nestandardní struktuře.
- **UPLOAD_IMAGE_PIXEL_LIMIT** — maximální počet pixelů nahrávaného obrázku (ochrana proti dekompresi velkých obrázků). Výchozí: 50 000 000.
- **ALLOWED_IMAGE_EXTENSIONS** — povolené přípony souborů. Výchozí: `{'jpeg', 'jpg', 'png', 'webp'}`.
- **ALLOWED_IMAGE_MIME_TYPES** — povolené MIME typy. Výchozí: `{'image/jpeg', 'image/png', 'image/webp'}`.

- **MAX_CONTENT_LENGTH** — maximální velikost nahrávaného souboru v bajtech. Výchozí: 16 MB (16 * 1024 * 1024).

Hashování hesel:

- **PASSWORD_HASHER_PARAMETERS** — slovník s parametry pro Argon2id. Výchozí: `time_cost=3, memory_cost=65536 (64 MB), parallelism=2, hash_len=32, salt_len=16`. Doporučení: výchozí parametry jsou bezpečné; při změně se existující hesla automaticky rehashují při příštím přihlášení.

Plánovač úloh (scheduler):

- **SCHEDULER_ENABLED** — zapne/vypne plánovač automatických úloh. Výchozí: True.
- **SCHEDULER_SESSION_CLEANUP_MINUTES** — interval čištění expirovaných sessions v minutách. Výchozí: 60.
- **SCHEDULER_UNUSED_IMAGES_CLEANUP_MINUTES** — interval mazání nepoužívaných obrázků. Výchozí: 180.
- **SCHEDULER_DISK_ORPHANS_CLEANUP_MINUTES** — interval mazání osiřelých souborů na disku. Výchozí: 720.
- **SCHEDULER_BACKUP_MINUTES** — interval automatického zálohování databáze v minutách. Výchozí: 30. Hodnota 0 zálohování vypne. Doporučení: 30–1440 (denní záloha) podle potřeby.

Zálohy:

- **BACKUP_DIR** — adresář pro ukládání záloh databáze. Výchozí: `instance/backups/`.
- **BACKUP_MAX_COUNT** — maximální počet uchovávaných záložních souborů (nejstarší se automaticky mažou). Výchozí: 48.

Ukázka doporučeného souboru instance/config.py pro produkční nasazení

Soubor config.py by měl obsahovat pouze hodnoty, které se liší od výchozích. Vše ostatní se automaticky načte z create_app(). Následující ukázka obsahuje položky, které je nutné nebo vhodné nastavit pro produkci:

```
# povinné:

SECRET_KEY = "vygenerujte_prikazem: python -c 'import secrets;
print(secrets.token_urlsafe(32))'"

DATABASE_CONNINFO = """
    dbname=cashier_app
    host=localhost
    user=postgres
    password=silne_heslo
    port=5432"""

# --- Reverzní proxy (nastavte podle vaší proxy, např. nginx) ---

# Například:
PROXY_FIX = {
    'x_for': 1,      # propustí 1 hodnotu z X-Forwarded-For -> skutečná
                    # klientská IP
    'x_proto': 1,    # vezme X-Forwarded-Proto -> http/https
    'x_host': 0,     # pokud potřebujete brát host z X-Forwarded-Host,
                    # nastavte 1
    'x_port': 0,     # většinou 0
    'x_prefix': 1,   # pokud nginx přidává X-Forwarded-Prefix
                    # (opakovaně), jinak 0
    'trusted_hosts': ['127.0.0.1', '::1'] # IP/hosty vašich proxy
}

SESSION_COOKIE_SECURE = True

READER_INFO = {
    'serial_port_options': {
```

```

        'baudRate': 9600,
        'dataBits': 8,
        'stopBits': 1,
        'parity': 'none',
        'flowControl': 'none'
    }
    # 'filters': [
    #     {'usbVendorId': 4292, 'usbProductId': 60000}
    # ]
}

# můžete přidat a upravit hodnoty pro:

# SESSION_COOKIE_SAMESITE = 'Lax'
# SESSION_ENFORCE_UA = False
# SESSION_ENFORCE_IP = False
# PASSWORD_HASHER_PARAMETERS = {
#     'time_cost': 3,
#     'memory_cost': 65536,
#     'parallelism': 2,
#     'hash_len': 32,
#     'salt_len': 16
# }
# MAX_UNDO_CHANGES = 30
# UNDO_TIME_LIMIT_MINUTES = 60
# REFUND_TIME_LIMIT_MINUTES = 5
# UPLOAD_FOLDER = os.path.join(app.instance_path, 'uploads',
# 'products')
# UPLOAD_IMAGE_PIXEL_LIMIT = 50_000_000
# ALLOWED_IMAGE_EXTENSIONS = {'jpeg', 'jpg', 'png', 'webp'}
# ALLOWED_IMAGE_MIME_TYPES = {'image/jpeg', 'image/png', 'image/webp'}
# MAX_CONTENT_LENGTH = 16 * 1024 * 1024 # 16MB
# SESSION_COOKIE_HTTPONLY = True
# SESSION_MAX_INACTIVE_DAYS = 7
# SCHEDULER_ENABLED = True

```

```
# SCHEDULER_SESSION_CLEANUP_MINUTES = 60
# SCHEDULER_UNUSED_IMAGES_CLEANUP_MINUTES = 180
# SCHEDULER_DISK_ORPHANS_CLEANUP_MINUTES = 720
# SCHEDULER_BACKUP_MINUTES = 30
# BACKUP_DIR = os.path.join(app.instance_path, 'backups')
# BACKUP_MAX_COUNT = 48
```

9 Databáze a její rozvržení

9.1 Schéma databáze

Obrázek 1 zobrazuje navržení databáze. Databáze obsahuje 17 tabulek, které lze rozdělit do několika logických skupin:

Identita a přístup:

- `employees` — zaměstnanci systému (administrátoři, manažeři, pokladní, prodejci),
- `users` — uživatelé/návštěvníci akcí,
- `sessions` — serverové sessions pro přihlášené zaměstnance,

Struktura akcí:

- `events` — akce/události,
- `booths` — stánky v rámci akce (typ `cashier` nebo `seller`),
- `products` — produkty s cenou a volitelným obrázkem,
- `categories` — kategorie produktů,
- `product_images` — metadata o obrázcích produktů.
- `product_images_failed_to_delete` — záznamy o obrázcích, které se nepodařilo smazat z úložiště.

Vazební tabulky:

- `employee_event_booth_roles` — přiřazení zaměstnanců k akcím a stánkům s definovanou rolí.
- `product_booth_link` — přiřazení produktů ke stánkům,

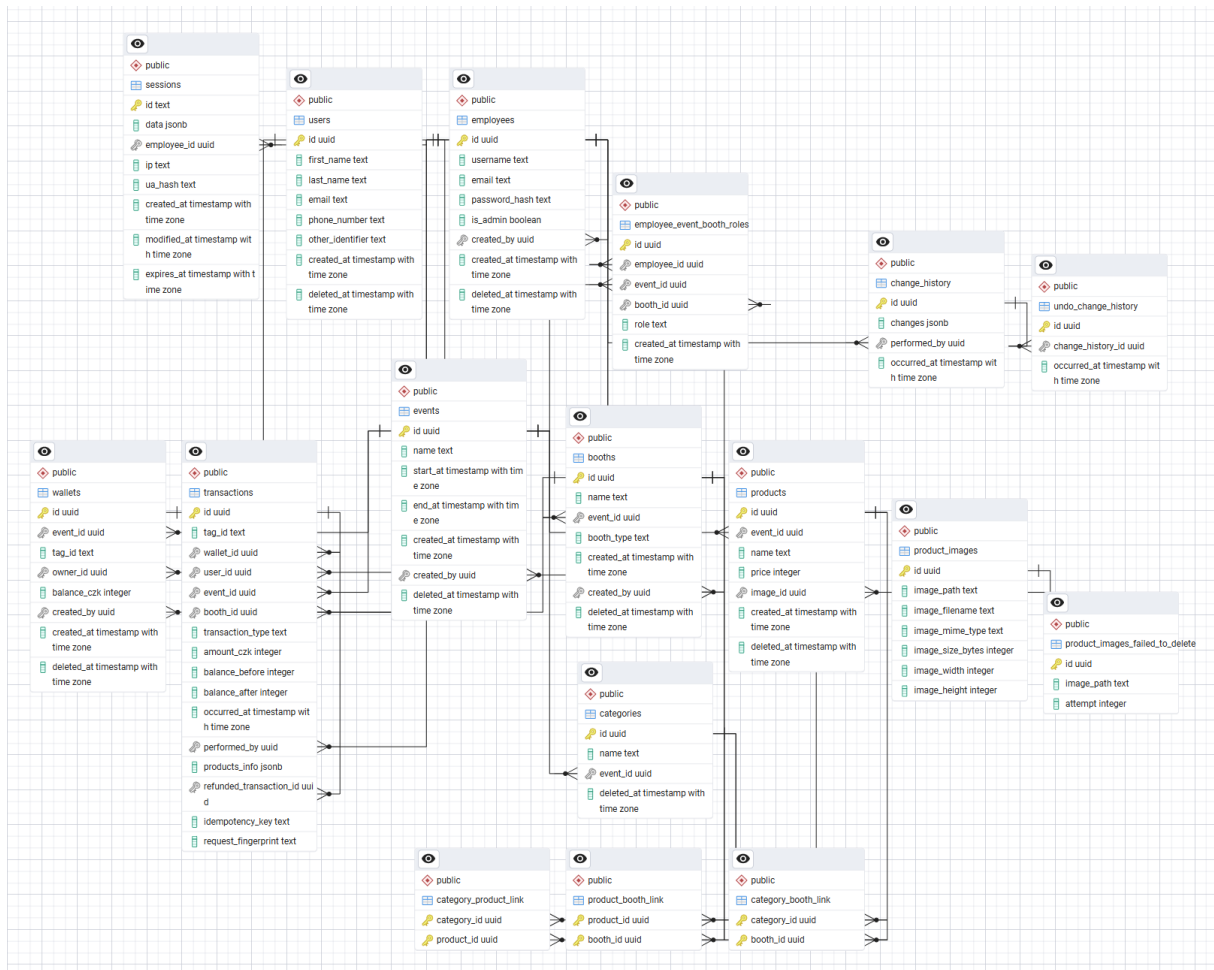
- `category_booth_link` — přiřazení kategorií ke stánkům,
- `category_product_link` — přiřazení produktů do kategorií.

Finance:

- `wallets` — virtuální peněženky vázané na kartu (tag), uživatele a akci,
- `transactions` — záznam o každé finanční operaci (neměnitelný), včetně podpory pro idempotenci a refundace.

Historie změn:

- `change_history` — záznam změn pro funkci undo,
- `undo_change_history` — záznamy o provedených undo operacích (pro funkci redo).



Obrázek 1 – ER diagram databáze
Zdroj: vlastní zpracování

9.2 Triggery a integritní omezení

Databáze využívá rozsáhlou soustavu triggerů, které zajišťují integritu dat přímo na úrovni databáze. Každá hlavní tabulka má `BEFORE` trigger, který:

- **Blokuje fyzické smazání** — příkaz `DELETE` je zachycen a převeden na `soft-delete` (nastavení `deleted_at = now()`). Tím je zaručeno, že žádný záznam nemůže být fyzicky odstraněn z databáze.
- **Chrání neměnné sloupce** — sloupce jako `created_at`, `created_by`, `event_id` nebo `booth_type` nelze po vytvoření záznamu měnit.
- **Normalizuje vstupní data** — automaticky odstraňuje mezery na začátku a konci textu, převádí e-maily na malá písmena a jména na formát s velkým prvním písmenem.

Trigger na tabulce `transactions` je nejkomplexnější — při vkládání nové transakce provádí více než deset kontrol (aktivita akce, existence stánku, oprávnění zaměstnance, dostatečný zůstatek, shoda `event_id` mezi peněženkou a stánkem atd.), zamyká řádek peněženky a aktualizuje zůstatek. U refundů navíc ověřuje, že transakce odkazuje na existující platbu prostřednictvím `refunded_transaction_id`, že odkazovaná transakce je typu `payment` a že dosud nebyla refundována. Trigger také kontroluje shodu typu transakce s typem stánku — `cashier` stánky smí provádět pouze `balance-change`, zatímco `seller` stánky pouze `payment` a `refund`. Zároveň kompletně blokuje `UPDATE` i `DELETE` operace na této tabulce, čímž zaručuje neměnnost finančních záznamů.

9.3 Indexy

Pro zajištění výkonu dotazů jsou v databázi definovány cílené indexy:

- **Transakce** — indexy na `wallet_id`, `(event_id, occurred_at DESC)` a `booth_id` pokrývají nejčastější dotazy pro historii a statistiky.
- **Peněženky** — unikátní parciální index na `(event_id, tag_id)` kde `deleted_at IS NULL` zajišťuje, že v rámci jedné akce existuje nejvýše jedna aktivní peněženka pro danou kartu.

- **Unikátní indexy** — parciální unikátní indexy (s podmínkou `WHERE deleted_at IS NULL`) zajišťují, že unikátní omezení platí pouze pro aktivní (nesmazané) záznamy. Například dva zaměstnanci mohou mít stejné uživatelské jméno, pokud je jeden z nich smazán.

10 Způsob programování a využití nástrojů

10.1 Struktura projektu a Flask blueprints

Aplikace je organizována jako Flask package s modulární strukturou. Vstupním bodem je tovární funkce `create_app()` v souboru `__init__.py`, která vytváří a konfiguruje Flask instanci.

Tento přístup se nazývá Application Factory pattern — místo vytvoření globální instance aplikace na úrovni modulu se Flask objekt vytváří uvnitř funkce. To přináší výhody jako snadnější testování (každý test může vytvořit vlastní instanci s odlišnou konfigurací), možnost spouštět více instancí vedle sebe a čistější oddělení konfigurace od kódu. Tento vzor byl zvolen podle doporučení oficiální dokumentace Flasku pro větší projekty.

Funkční logika je rozdělena do samostatných modulů, z nichž každý registruje jeden nebo více blueprintů:

```
cashier_app/
├── __init__.py          # Tovární funkce, konfigurace, rate
limiter
├── auth.py              # Autentizace (login/logout)
├── db.py                 # Connection pool
├── deleted.py            # Stránky smazaných záznamů (uživatelé,
akce)
├── employee_events_booths.py # API: výběr aktivní akce a stánku v
session
├── employees.py          # API + stránka: zaměstnanci
├── errors.py             # Vlastní výjimky aplikace
└── index.py              # Úvodní stránka
```

─ paste.py	# Kopírování/klonování
─ pg_session.py	# Serverové sessions v PostgreSQL
─ reader_info.py	# API: informace o čtečce
─ scheduler.py	# Plánovač úloh na pozadí (APScheduler)
─ session_api.py	# API: informace o aktuální relaci
─ settings.py	# Stránka + API: nastavení profilu
zaměstnance	
─ transactions.py	# API: platby, dobíjení, refundace
─ undo_and_redo.py	# Zpět/znovu
─ users.py	# API: správa uživatelů
─ wallets.py	# API: správa peněženek
─ schema.sql	# Databázové schéma
─ development_values.sql	# Vývojová testovací data
─ events/	
─ __init__.py	# CRUD akce, kaskádové mazání/obnovení
─ booths.py	# Stánky
─ categories.py	# Kategorie
─ event_employees.py	# Role zaměstnanců v akcích
─ products.py	# Produkty + nahrávání obrázků
─ statistics.py	# Statistiky akcí
─ transaction_history.py	# Historie transakcí (akce, uživatelé)
─ utils/	
─ cascade_capture.py	# Kaskádové zachycení dat pro undo/redo
─ employees_users.py	# Validace údajů zaměstnanců a
uživatelů	
─ events.py	# Validace názvů akcí a stánků
─ general.py	# Obecné pomocné funkce (UUID, IP,
zámky)	
─ images.py	# Správa obrázků produktů
─ link_sync.py	# Synchronizace vazebních tabulek
(undo/redo)	
─ products.py	# Validace produktů/kategorií
─ query_builder.py	# Generátor SQL dotazů
─ transactions.py	# Idempotentní vkládání transakcí
─ static/	# JS, CSS, ikony, obrázky
─ templates/	# Jinja2 HTML šablony

Moduly typicky definují dva blueprinty — jeden pro HTML stránky (s URL prefixem jako `/events`) a jeden pro API endpointy (s prefixem `/api/events`). Toto oddělení usnadňuje orientaci v kódu a umožňuje nezávislý vývoj frontend³ a backend⁴ částí.

10.2 Připojení k databázi — psycopg a connection pool

Pro komunikaci s PostgreSQL je použita knihovna psycopg 3 s connection poolem. Connection pool udržuje předem vytvořená připojení k databázi a přiděluje je jednotlivým požadavkům, čímž eliminuje režii opakovaného navazování spojení:

```
def init_app(app: Flask):
    conninfo = app.config.get('DATABASE_CONNINFO')
    pool = ConnectionPool(
        conninfo,
        kwargs={'row_factory': dict_row},
        min_size=1,
        max_size=5,
        timeout=30,
        open=True)
    app.extensions['db_pool'] = pool
```

Parametr `row_factory=dict_row` zajišťuje, že výsledky dotazů jsou vráceny jako slovníky (s názvy sloupců jako klíče), nikoli jako tuply.

Pool je uložen v `app.extensions` a přístupný přes funkci `get_pool()`.

```
def get_pool() -> ConnectionPool:
    pool: ConnectionPool = current_app.extensions.get('db_pool')
    if pool is None:
        raise RuntimeError("db pool not initialized")
    return pool
```

³ Frontend je část aplikace, se kterou pracuje uživatel v prohlížeči.

⁴ Backend je serverová část aplikace, která zpracovává data a logiku.

Připojení se získává voláním `get_pool().connection()` jako context manager (příkaz `with`). Po opuštění bloku `with` se připojení automaticky vrátí zpět do poolu pro další požadavky — není tedy nutné ho ručně zavírat:

```
sql, query_params = build_insert_statement('events', params,
returning='*')
try:
    with get_pool().connection() as conn:
        with conn.cursor() as cur:
            new_event = cur.execute(sql, query_params).fetchone()
            save_change(cur, [{
                'table': 'events',
                'old_values': None,
                'new_values':
convert_dict_to_serializable(dict(new_event))
            }], g.employee['id'])
except IntegrityError as e:
    constraint = get_constraint_name(e)
    if constraint == 'unique_index_events_name_active':
        return jsonify(error='event_name_taken'), 409
    return jsonify(error='db_integrity_error'), 400
return jsonify(), 200
```

Psycopg 3 používá ve výchozím nastavení transakční model — každé připojení získané z poolu automaticky zahájí transakci. Pokud blok `with` skončí úspěšně, transakce se potvrdí (COMMIT). Pokud dojde k výjimce, transakce se automaticky odvolá (ROLLBACK). Díky tomu je zaručena atomicita operací bez nutnosti explicitního řízení transakcí.

Veškeré SQL dotazy používají parametrizované zápisy (`%s` placeholder), nikdy se nepoužívá formátování řetězců. Pro dynamické sestavování dotazů slouží modul `query_builder.py`, který generuje INSERT, UPDATE a DELETE příkazy s parametry.

10.3 Serverové sessions v PostgreSQL

Místo výchozího Flask mechanismu (cookie-based sessions) implementuje projekt vlastní session backend ukládající data do PostgreSQL tabulky `sessions`. Třída `PgSessionInterface` implementuje rozhraní `SessionInterface` a zajišťuje:

- Generování bezpečného session ID pomocí `secrets.token_urlsafe(32)`.
- Ukládání session dat jako JSONB v databázi.
- Volitelnou regeneraci session ID při přihlášení (ochrana proti session fixation).
- Volitelné vynucování shody IP adresy a User-Agenta pro každý požadavek.
- Automatické čištění expirovaných sessions na pozadí pomocí APScheduleru.

10.4 Zpracování chyb — vlastní výjimky

Aplikace definuje vlastní výjimky v modulu `errors.py`, které umožňují přesné rozlišení chybových stavů a jejich převod na odpovídající HTTP odpovědi. Mezi klíčové výjimky patří:

- **InsufficientBalanceError** — nedostatečný zůstatek peněženky pro provedení transakce (HTTP 400).
- **IdempotencyKeyDataConflict** — opakovaný požadavek se stejným idempotentním klíčem, ale odlišnými parametry (HTTP 409).
- **NoRowsAffectedError** — operace neovlivnila žádný řádek, typicky znamená, že záznam neexistuje (HTTP 404).
- **MultipleRowsAffectedError** — operace ovlivnila více řádků, než se očekávalo — indikátor logické chyby, logováno jako výjimka (HTTP 500).
- **CanNotDeleteLastAdminError** — pokus o smazání posledního administrátora v systému (HTTP 400).
- **PgTryAdvisoryLockError** — nepodařilo se získat advisory lock, typicky proto, že stejný zaměstnanec již provádí jinou operaci (HTTP 409).
- **ForbiddenError** — zaměstnanec nemá oprávnění k požadované akci (HTTP 403).

- **UndoTargetDeletedError** — pokus o undo aktualizace, ale cílová entita byla mezitím smazána jiným uživatelem — řešeno jako varování, nikoliv chyba.

Každý API endpoint zachytává tyto výjimky v bloku try-except a převádí je na JSON odpověď s klíčem error a příslušným HTTP kódem.

Kromě výjimek na úrovni aplikační logiky registruje Flask dva globální error handlers:

- **RequestEntityTooLarge (413)** — nahrávaný soubor překračuje maximální velikost.
- **Too Many Requests (429)** — překročen limit požadavků (rate limiting).

10.5 Generátor SQL dotazů (Query Builder)

Pro opakující se CRUD operace slouží modul `query_builder.py`, který na základě názvu tabulky a slovníku parametrů generuje parametrizované SQL příkazy. Tento přístup snižuje množství opakujícího se kódu a zároveň zachovává bezpečnost parametrizovaných dotazů:

```
sql, params = build_insert_statement(
    'events',
    {'name': 'název', 'created_by': 1},
    returning='*'
)
# Výsledek (jako string): INSERT INTO events (name, created_by)
#           VALUES (%s, %s) RETURNING *
# Doopravdy vypadá: Composed([SQL('\n    INSERT INTO '), Identifier...
# params: ['název', 1]
```

Query builder podporuje i pokročilé operace jako `ON CONFLICT DO NOTHING` (pro idempotentní vkládání) a soft-delete (generuje `UPDATE ... SET deleted_at = now() místo DELETE`).

10.6 Synchronizace vazebních tabulek

Systém obsahuje řadu vazebních tabulek typu many-to-many (`product_booth_link`, `category_booth_link`, `category_product_link`, `employee_event_booth_roles`), které propojují produkty se stánky, kategorie se stánky a produkty, a zaměstnance s rolemi v akcích. Při editaci přiřazení (například „tento produkt patří ke stánkům A, B a C“) je potřeba synchronizovat vazební tabulku s novým požadovaným stavem.

Modul `link_sync.py` implementuje pro každou vazební tabulku synchronizační funkci, která pracuje ve čtyřech krocích:

1. Načte aktuální vazby z databáze.
2. Smaže všechny existující vazby pro danou entitu.
3. Vloží nové vazby podle požadovaného stavu.
4. Porovná starý a nový stav a vrátí pouze diff (přidané a odebrané vazby) jako změnové záznamy pro systém undo.

Díky kroku 4 se do `change_history` nezapisují vazby, které zůstaly beze změny — ukládá se pouze to, co se skutečně změnilo. Celá synchronizace probíhá v rámci jedné databázové transakce, takže je buď provedena kompletně, nebo vůbec.

11 Práva přístupu

11.1 Hierarchie rolí

Systém implementuje čtyřúrovňovou hierarchii oprávnění, kterou znázorňuje Tabulka 1.

Tabulka 1 – Přehled rolí a jejich oprávnění
Zdroj: vlastní zpracování

Role	Rozsah	Oprávnění
Admin	Globální	Vytváření zaměstnanců, akcí, správa smazaných záznamů, kopírování, přístup ke všem akcím a stánkům
Event manager	V rámci akce	Správa stánků, produktů, kategorií, zaměstnanců akce, zobrazení statistik a historie, přístup ke všem stánkům akce
Cashier	V rámci stánku	Správa uživatelů, vytváření/vracení peněženek, dobíjení/výběr prostředků
Seller	V rámci stánku	Provádění plateb a refundací

Role admina je uložena přímo v tabulce `employees` (sloupec `is_admin`). Ostatní role jsou definovány v tabulce `employee_event_booth_roles`, kde je zaměstnanec přiřazen ke konkrétní akci a případně konkrétnímu stánku. Event manager má `booth_id = NULL` (vztahuje se k celé akci), zatímco cashier a seller mají přiřazen konkrétní stánek.

11.2 Vynucování oprávnění

Oprávnění jsou vynucována na dvou nezávislých úrovních:

1. **Aplikační vrstva** — každý API endpoint na začátku ověří, zda je zaměstnanec přihlášen a zda má dostatečnou roli. Například endpoint pro vytvoření akce kontroluje `logged_employee['is_admin']`, endpoint pro editaci produktu ověřuje funkci `is_manager()`.
2. **Databázová vrstva** — trigger na tabulce `transactions` nezávisle ověřuje, zda zaměstnanec provádějící transakci má odpovídající roli pro daný stánek.

Toto dvojité ověření zajišťuje, že i v případě chyby v aplikační logice nemůže neoprávněný zaměstnanec provést finanční operaci.

11.3 Vzájemná výlučnost rolí

Databázový trigger na tabulce `employee_event_booth_roles` zajišťuje, že pokud je zaměstnanec event managerem dané akce (má přiřazení s `booth_id = NULL`), nemůže být zároveň přiřazen ke konkrétnímu stánku téže akce — a naopak. Toto omezení zabraňuje konfliktům v logice oprávnění.

12 Čtení karet pomocí čteček

12.1 Komunikace přes Web Serial API

Čtení RFID/NFC karet je implementováno na straně klienta pomocí Web Serial API. Celý proces probíhá v několika krocích:

1. **Výběr portu** — aplikace nejprve zkontroluje již spárované porty (`navigator.serial.getPorts()`). Pokud je spárován právě jeden port, použije se automaticky. V opačném případě je uživatel vyzván k výběru čtečky prostřednictvím systémového dialogu (`navigator.serial.requestPort()`).
2. **Otevření portu** — port se otevře s parametry definovanými v konfiguraci serveru (typicky `baudRate: 9600, dataBits: 8, stopBits: 1, parity: 'none'`). Tyto parametry se načítají z API endpointu `/api/reader/info`.
3. **Čtení dat** — binární proud dat ze čtečky je převeden na textový proud pomocí `TextDecoderStream`. Data se čtou znak po znaku a akumulují do řetězce, dokud není detekován konec ID karty znakem (`\n` nebo `\r`) nebo 100ms od posledního přečtení:

```

async function readStringStreamReader(reader, readableStreamClosed,
onCardRead) {
  let timeoutId;
  try {
    let cardId = '';
    while (true) {
      const { value, done } = await reader.read();
      clearTimeout(timeoutId);
      if (done) {
        break;
      }
      cardId += value;
      // konec id karty, nejspíš \n nebo \r
      if (cardId.includes('\n') || cardId.includes('\r')) {
        onCardRead(cardId);
        cardId = '';
      } else {
        // nebo dost dlouho od posledního přečtení (100ms)
        timeoutId = setTimeout(() => {
          onCardRead(cardId);
          cardId = '';
        }, 100);
      }
    }
  } catch (error) {
    console.warn('Chyba čtení:', error);
  } finally {
    clearTimeout(timeoutId);
    reader.releaseLock();
    await readableStreamClosed.catch(() => { });
  }
}

```

4. **Zpracování ID** — po načtení kompletního ID karty (například 00A713A700000000) je vyvolán callback, který vyhledá peněženku s odpovídajícím `tag_id` a zobrazí informace o uživateli.

13 ACID operace a práce s databází při finančních operacích

13.1 Průběh platby

Platba u prodejního stánku, kterou zobrazuje Příloha 2, probíhá v těchto krocích:

1. **Klient** — zaměstnanec přiloží kartu návštěvníka ke čtečce, vybere produkty a potvrdí platbu. Klient vygeneruje unikátní idempotentní klíč

(`crypto.randomUUID()`) a odešle požadavek na API.

2. **Server — validate** — server ověří přihlášení zaměstnance, existenci a aktivitu akce, oprávnění pro daný stánek, formát vstupních dat a vyhledá peněženku podle `tag_id`.

3. **Server — vložení transakce** — v rámci jedné databázové transakce se provede:

- Výpočet SHA-256 otisku (fingerprint) ze všech parametrů požadavku.
- Pokus o vložení záznamu do tabulky `transactions` s klauzulí `ON CONFLICT (idempotency_key) DO NOTHING`.
- Pokud je záznam vložen, databázový trigger zamkne peněženku (`FOR UPDATE`), ověří dostatečný zůstatek, vypočítá `balance_before/balance_after` a aktualizuje zůstatek peněženky.
- Pokud vložení selže kvůli duplicitnímu klíči, server porovná fingerprint — shoda znamená idempotentní opakování (úspěch), neshoda znamená konflikt dat (chyba 409).

4. **Klient — zpracování odpovědi** — při úspěchu se aktualizuje zobrazení zůstatku, při chybě se zobrazí odpovídající hlášení.

13.2 Idempotence a fingerprint

Mechanismus idempotence chrání proti duplicitním transakcím. Klient generuje unikátní klíč pro každou zamýšlenou platbu a odesílá ho v HTTP hlavičce

`Idempotency-Key`. Server tento klíč ukládá společně s SHA-256 otiskem všech parametrů transakce:

```
fingerprint_source = json.dumps(
    {key: convert_uuids_to_str(value) for key, value in
    fingerprint_cols.items()},
    separators=(',', ':'), sort_keys=True)
request_fingerprint =
hashlib.sha256(fingerprint_source.encode('utf-8')).hexdigest()
```

Při opakovaném požadavku se stejným klíčem server rozpozná, že transakce již byla provedena. Pokud se fingerprint⁵ shoduje, jde o legitimní opakování (například kvůli výpadku sítě). Pokud se fingerprint liší, jde o pokus o zneužití klíče pro jinou transakci — server vrátí chybu.

13.3 Refundace

Refundace (vrácení platby), která je také zobrazena v Příloze 2, je speciální typ transakce, který vytvoří nový záznam s opačnou částkou a odkazem na původní transakci (`refunded_transaction_id`). Systém rozlišuje dva typy refundace:

- **Refundace seller stánku** — dostupná přímo z rozhraní prodejního stánku. Umožňuje refundovat pouze posledně provedenou platbu na dané peněženice, a to pouze v konfigurovatelném časovém limitu (výchozí: 5 minut). Podmínka pro nalezení refundovatelné transakce ověřuje, že transakce dosud nebyla refundována.
- **Administrátorská refundace** — dostupná z historie transakcí uživatele i z celkové historie transakcí akce. Umožňuje administrátorovi nebo správci akce příslušné akce refundovat libovolnou dosud nerefundovanou platbu bez

⁵ Fingerprint slouží jako kontrolní otisk požadavku; díky němu lze rozlišit skutečné opakování stejné operace od pokusu použít stejný idempotentní klíč pro jiná data.

časového limitu. Endpoint POST /api/transactions/admin-refund nejprve ověří oprávnění (admin nebo event manager s booth_id IS NULL v tabulce employee_event_booth_roles), poté zkontroluje, že platba existuje a ještě nebyla refundována, a provede refundaci pomocí stejného mechanismu make_transaction.

13.4 Zajištění integrity na úrovni databáze

Kromě aplikační logiky zajišťuje integritu finančních dat řada databázových mechanismů:

- **CHECK constraint** `balance_after = balance_before + amount_czk` — matematická kontrola, že nový zůstatek odpovídá starému plus částce transakce.
- **Blokace UPDATE/DELETE** na tabulce `transactions` — trigger vyvolá výjimku při jakémkoliv pokusu o změnu nebo smazání transakce.
- **Row-level lock** (`SELECT ... FOR UPDATE`) — zamezuje souběžné modifikaci stejné peněženky.
- **Unikátní index** na `idempotency_key` — zabraňuje vložení dvou transakcí se stejným klíčem.

14 Caching

14.1 Serverový caching statických souborů

Aplikace implementuje verzování statických souborů (CSS, JS) pomocí MD5 hashů. Funkce `versioned_static()` vypočítá hash obsahu souboru a připojí ho jako query parametr k URL:

```
def versioned_static(filename):
    if filename not in _static_hash_cache:
        filepath = os.path.join(app.static_folder, filename)
        try:
```

```

        with open(filepath, 'rb') as f:
            _static_hash_cache[filename] =
hashlib.md5(f.read()).hexdigest()[ :10]
        except FileNotFoundError:
            _static_hash_cache[filename] = '0'
        return f'/static/{filename}?v={_static_hash_cache[filename]}'

```

Soubory s verzovacím parametrem dostávají hlavičku `Cache-Control: max-age=31536000` (1 rok), protože při změně obsahu se změní hash a tím i URL — prohlížeč automaticky stáhne novou verzi. Neverzované soubory (JS moduly importované jinými moduly) dostávají `Cache-Control: no-cache`, aby se vždy revalidovaly.

14.2 Klientský caching dat

Na straně klienta implementuje modul `cache_factory.js` generickou cache pro asynchronní funkce. Tovární funkce `cacheFunctionFactory` obalí libovolnou async funkci a vrátí cachovanou verzi s automatickým obnovováním na pozadí:

```

export function cacheFunctionFactory(func, cacheTimeMs = 1000 * 60 * 2
/*2 minuty*/, cacheRefetchMs = 1000 * 60 /*1 minuta*/) {
    const cache = {
        data: null,
        fetchTime: 0
    }
    let promiseHolder;

    const wrapperFunc = (noCache = false, ...args) => {
        if (!noCache && cache.data && cache.fetchTime + cacheTimeMs >
Date.now()) { // vrátí cache
            if (cache.fetchTime + cacheRefetchMs < Date.now()) {
                // refetch na pozadí (nečekej)
                wrapperFunc(true, ...args);
            }
            return Promise.resolve(cloneData(cache.data));
        }
    }
}

```



```

    }
    // aktuálně se získávají data
    if (promiseHolder) return promiseHolder;

    // získání dat vloženou funkcí
    promiseHolder = (async () => {
        try {
            cache.data = await func(...args);
            cache.fetchTime = Date.now();

            return cloneData(cache.data);

        } finally {
            promiseHolder = null;
        }
    })();

    return promiseHolder;
};

const resetCacheFunc = () => {
    cache.data = null;
    cache.fetchTime = 0;
    wrapperFunc().catch(() => { });
};

return [wrapperFunc, resetCacheFunc];
}

```

Cache funguje ve třech režimech:

- **Cache hit (čerstvá data)** — data jsou mladší než `cacheRefetchMs` → vrátí se okamžitě klon dat z cache.
- **Cache hit (stará data)** — data jsou starší než `cacheRefetchMs`, ale mladší než `cacheTimeMs` → vrátí se klon z cache a na pozadí se spustí obnovení.

- **Cache miss** — data nejsou v cache nebo jsou starší než `cacheTimeMs` → provede se nový fetch.

Deduplikace požadavků zabraňuje souběžnému odesílání více požadavků na stejná data — pokud je fetch již v běhu (`promiseHolder`), další volání čekají na výsledek prvního požadavku.

Cache se používá pro produkty, uživatele, peněženky a akce. Každý modul exportuje funkci `resetCache()`, která vymaže cache a spustí nový fetch na pozadí — volá se po operacích, které modifikují data (vytvoření, editace, smazání).

14.3 Persistence stavu objednávky

Třída `Order` na straně klienta ukládá aktuální stav objednávky (košíku) do `sessionStorage` prohlížeče. Díky tomu objednávka přežije obnovení stránky (refresh) v rámci stejné záložky, ale automaticky se vymaže po zavření záložky.

15 Statistiky a historie plateb

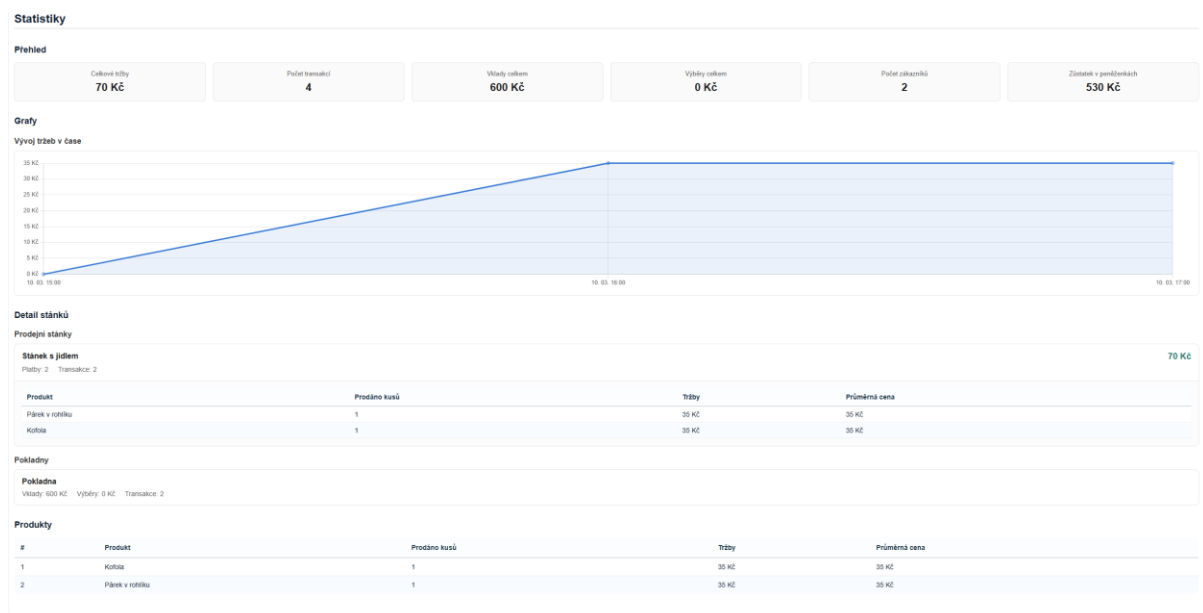
15.1 Statistiky akcí

Obrázek 2 zobrazuje statistický přehled akce v rozhraní správy akce. Data poskytuje endpoint `GET /api/events/<event_id>/statistics` a frontend je vykresluje do čtyř vizuálních sekcí:

- **Přehled** — šest informačních karet zobrazujících celkové tržby, počet transakcí, vklady celkem, výběry celkem, počet zákazníků (unikátních uživatelů) a celkový zůstatek v peněženkách.
- **Graf vývoje tržeb** — čárový graf (Chart.js) zobrazující hodinový průběh tržeb. Data se agregují pomocí `DATE_TRUNC('hour', occurred_at)`.
- **Detail stánků** — stánky jsou rozděleny podle typu. U prodejních stánků se zobrazují tržby, počet plateb a transakcí; kliknutím na stánek se rozbalí tabulka s prodanými produkty (název, prodané kusy, tržby, průměrná cena). U pokladen se zobrazují vklady, výběry a počet transakcí.

- **Produkty** — tabulka produktů seřazených podle tržeb, s údaji o prodaném množství, tržbách a průměrné ceně. Data o produktech se extrahují z JSONB sloupce products_info v tabulce transactions pomocí funkce jsonb_array_elements.

Všechny statistické dotazy vylučují refundované transakce dvojí podmínkou — transaction_type != 'refund' AND NOT EXISTS (SELECT 1 FROM transactions r WHERE r.refunded_transaction_id = t.id) — čímž se odfiltrují jak samotné refundace, tak původní transakce, které byly refundovány.



Obrázek 2 – Přehled statistik akce
Zdroj: vlastní zpracování

15.2 Historie transakcí

Aplikace poskytuje dva typy historie transakcí:

- **Historie transakcí uživatele** — zobrazuje všechny transakce konkrétního uživatele v rámci akce, včetně názvu stánku, jména zaměstnance, částek, zůstatků a informací o produktech. Přístupná z pohledu pokladního (cashier) i manažera akce (viz 3).
- **Historie transakcí akce** — zobrazuje kompletní výpis všech transakcí celé akce. Přístupná pouze pro event managery a adminy.

Obě historie jsou zobrazeny na dedikovaných HTML stránkách s tabulkovým zobrazením. Pokud má přihlášený zaměstnanec oprávnění provádět administrátorskou refundaci (admin nebo event manager dané akce), zobrazí se v tabulce transakcí sloupec Akce s tlačítkem Refundovat u každé dosud nerefundované platby. Při tisku jsou tlačítka a sloupec skryta pomocí CSS (@media print).

Historie transakcí

JMÉNO	PŘÍJMENÍ	EMAIL	TELEFON	JINÝ IDENTIFIKÁTOR
Ondřej	Procházka	ondrej.prochazka@student.mgvsetin.cz	-	-
AKCE				
Zahradní slavnost 2026				

CELKOVÝ POČET TRANSAKČÍ	VKLADY	VÝBĚRY	VŠECHNY PLATBY	NEVRÁCENÉ PLATBY	VRÁCENÍ
6	500 Kč	0 Kč	-244 Kč	-155 Kč	89 Kč
	1 transakce	0 transakcí	4 transakce	3 transakce	1 transakce
CELKOVÝ ZŮSTATEK					
345 Kč					
1 karta					

Karty uživatele

#	ID karty	Vklady	Výběry	Platby	Vrácení	Celkem transakcí	Konečný zůstatek
1	00C935C900000000	500 Kč transakce: 1x	0 Kč transakce: 0x	-244 Kč transakce: 4x	89 Kč transakce: 1x	6	345 Kč

Všechny transakce

#	Datum a čas	ID karty	Typ	Částka (Kč)	Stav před	Stav po	Stánek	Zaměstnanec	Produkty
1	10. 3. 2026 12:31:46	00C935C900000000	Vklad	+500	0	500	Pokladna vchod	vojtěch_vyroubal	-
2	10. 3. 2026 14:31:46	00C935C900000000	Platba	-89	500	411	Občerstvení	josef_kovář	Hamburger (1× 89 Kč)
3	10. 3. 2026 14:46:46	00C935C900000000	Platba	-55	411	356	Grill	kateřina_haganová	Hranolky (1× 55 Kč)
4	10. 3. 2026 15:16:46	00C935C900000000	Vrácení	+89	356	445	Občerstvení	josef_kovář	Hamburger (1× 89 Kč)
5	10. 3. 2026 16:01:46	00C935C900000000	Platba	-65	445	380	Občerstvení	josef_kovář	Klobása (1× 65 Kč)
6	10. 3. 2026 17:26:46	00C935C900000000	Platba	-35	380	345	Občerstvení	Štěpán_vrána	Kofola (1× 35 Kč)

Obrázek 3 – Stránka historie transakcí uživatele
Zdroj: vlastní zpracování

16 Kopírování (paste) a zpět/znovu (undo/redo)

16.1 Kopírování (paste)

Endpoint `POST /api/paste` umožňuje klonování entit v rámci systému. Jedná se o operaci dostupnou pouze administrátorům (pro klonování zaměstnanců, akcí

a čehokoliv v rámci akce) a event manažerům, kteří však můžou klonovat pouze věci, ke kterým mají přístup. Podporované operace:

- **Klonování zaměstnanců** — zkopíruje zaměstnance včetně přiřazení rolí. Generuje unikátní uživatelské jméno a e-mail přidáním suffixu `_copy`, `_copy2` atd.
- **Klonování akcí** — vytvoří novou akci se všemi stánky, produkty, kategoriemi a vazebnými tabulkami.
- **Klonování stánků** — zkopíruje stánek a obsah stánku (produkty, kategorie, přiřazení zaměstnanců) do cílových stánků nebo nových akcí. Pokud se kopíruje stánek do jeho akce, tak se nevytvoří nové produkty, ale pouze se přiřadí.
- **Klonování produktů a kategorií** — jednotlivé nebo hromadné kopírování.
- **Klonování přiřazení zaměstnanců**

Pro výběr věcí ke kopírování lze používat myš nebo šipky spolu s ctrl nebo shift klávesami. Po vybrání lze zkopírovat pomocí ctrl + c a vložit pomocí ctrl + v. Pokud je kopírována akce, nová akce se vytvoří vložením v manažeru akcí. Jinak se může kopírovat do akcí nebo stánků jejich vybráním (stejně jako u kopírování) nebo otevřením manažeru akce.

Všechny vazební tabulky se vkládají s klauzulí `ON CONFLICT DO NOTHING`, aby nedošlo k chybě při duplicitních vazbách. Celá operace probíhá v rámci jedné databázové transakce a je chráněna advisory lockem (`pg_try_advisory_xact_lock`) proti souběžnému provádění.

Každá operace paste ukládá změny do `change_history`, takže ji lze vrátit zpět pomocí undo.

16.2 Vrátit zpět a provést znovu (undo/redo)

Systém implementuje generický mechanismus undo/redo (ctrl + z/y) pro všechny CRUD operace nad správou akcí, stánků, produktů, kategorií a zaměstnanců. Finanční transakce undo nepodléhají — pro vrácení platby slouží refundace.

Princip fungování:

Každá operace, která mění data (vytvoření, editace, smazání), volá funkci `save_change()`, která zapíše do tabulky `change_history` JSON popis změny:

```
save_change(cur, [{
    'table': 'products',
    'old_values': {'id': '...', 'name': 'Hamburger',
                  'price': 50},
    'new_values': {'id': '...', 'name': 'Cheeseburger',
                  'price': 65}
}], employee_id)
```

Konvence: `old_values=None` znamená INSERT (nový záznam),
`new_values=None` znamená DELETE, obojí nastavené znamená UPDATE.

Kaskádové zachycení dat při mazání:

Smazání entity s vazbami (například celé akce) ovlivňuje řadu závislých záznamů — stánky, produkty, kategorie, vazební tabulky, přiřazení zaměstnanců i peněženky.

Aby bylo možné takovou operaci kompletně vrátit zpět, používá systém modul `cascade_capture.py`, který před smazáním zachytí všechna dotčená data do jednoho změnového záznamu. Například funkce `capture_event_cascade()`

postupně načte událost, všechny její stánky, produkty, kategorie, záznamy ve vazebních tabulkách (`product_booth_link`, `category_booth_link`, `category_product_link`), role zaměstnanců a peněženky — a pro každý záznam vytvoří změnový objekt s `old_values` a `new_values=None` (indikace smazání).

Analogické funkce existují pro stánky (`capture_booth_cascade`), produkty (`capture_product_cascade`), kategorie (`capture_category_cascade`) a zaměstnance (`capture_employee_cascade`). Všechny zachycené změny se uloží jako jedna atomická operace do `change_history`, takže undo obnoví celý strom závislostí najednou.

Undo — nalezne nejnovější nezrušenou změnu daného zaměstnance a aplikuje ji v opačném směru (INSERT → smazání, DELETE → obnovení, UPDATE → obnova starých hodnot). Změny se aplikují ve správném pořadí — nejprve obnovení smazaných rodičů, poté aktualizace, nakonec smazání vložených dětí.

Redo — nalezne nejnověji zrušenou změnu (záznam v `undo_change_history`) a znovu ji aplikuje. Pořadí je opačné oproti undo.

Bezpečnostní opatření:

- Advisory lock zabraňuje souběžnému undo/redo stejným zaměstnancem.
- Konfigurovatelný časový limit (výchozí: 60 minut) a maximální počet kroků (výchozí: 30).
- Detekce konfliktů — pokud byla provedena změna jiným administrátorem nebo manažerem, která brání operaci (např. obnovení produktu, když byl vytvořen nový produkt se stejným jménem), operace nelze provést a ukáže se informace o konfliktu.
- Po každé operaci se provedou vazební kontroly — před obnovením vazebního záznamu se ověří, že odkazované entity stále existují.

17 Využití — jak vypadá používání aplikace

17.1 Přihlášení

Zaměstnanec se přihlásí uživatelským jménem nebo e-mailem a heslem na přihlašovací stránce. Po úspěšném přihlášení je přesměrován na hlavní stránku.

17.2 Výběr akce a stánku

Po přihlášení si zaměstnanec vybere akci, ke které má přiřazenou roli. Podle typu role se mu zobrazí odpovídající rozhraní:

- **Admin** — vidí všechny akce, může přejít do správy akcí, zaměstnanců nebo nastavení.
- **Event manager** — vidí akce, kde je manažerem, a může je spravovat.
- **Cashier/Seller** — po výběru akce si vybere stánek a přejde do pokladního rozhraní.

17.3 Pokladní rozhraní (index)

Hlavní pracovní stránka slouží pro obsluhu zákazníků:

- **Načtení karty** — přiložením karty ke čtečce se automaticky identifikuje peněženka a zobrazí aktuální zůstatek.
- **Cashier (pokladní) stánek** — zaměstnanec může vytvářet, upravovat a mazat uživatele, vytvářet a vracet peněženky, dobíjet a vybírat prostředky. Zobrazuje se seznam uživatelů s možností vyhledávání (viz Obrázek 4).

The screenshot displays the cashier interface. On the left, there is a table titled 'Uživatelé' (Users) with columns for ID, Name, Surname, Email, Phone Number, Other Identifier, and Card. Five users are listed. On the right, there is a form titled 'Karta' (Card) with a search bar for 'Uživatel' (User) and fields for Name, Surname, Email, Phone Number, Other Identifier, and Card balance. A 'Zrušit' (Cancel) button and a 'Vytvořit a přiřadit kartu' (Create and assign card) button are at the bottom.

#	Jméno	Příjmení	Email	Telefonní číslo	Jiný identifikátor	Karty
1	Erik	Muijsenberg	erik.muijsenberg@gsi.cz	-	-	661 Kč (005AC0280...)
2	Ondřej	Procházka	ondrej.prochazka@student.mgvset...	-	-	345 Kč (00C935C90...)
3	Peter	Pincosy	-	-	Učitel angličtiny na Masarykově gy...	-
4	Thomas	Muijsenberg	thomas.muijsenberg@student.mgv...	+420775937283	-	500 Kč (newcard1)
5	Tobiáš	Filgas	tobias.filgas@student.mgvsetin.cz	-	-	241 Kč (00A713A70...)

Karta
ID: 00B824B800000000
Zůstatek: -

Uživatel

Jméno

Příjmení

Email

Telefonní číslo

+33

Jiný identifikátor

Přidat nebo ubrat peníze (Kč)

0

Nový zůstatek (Kč)

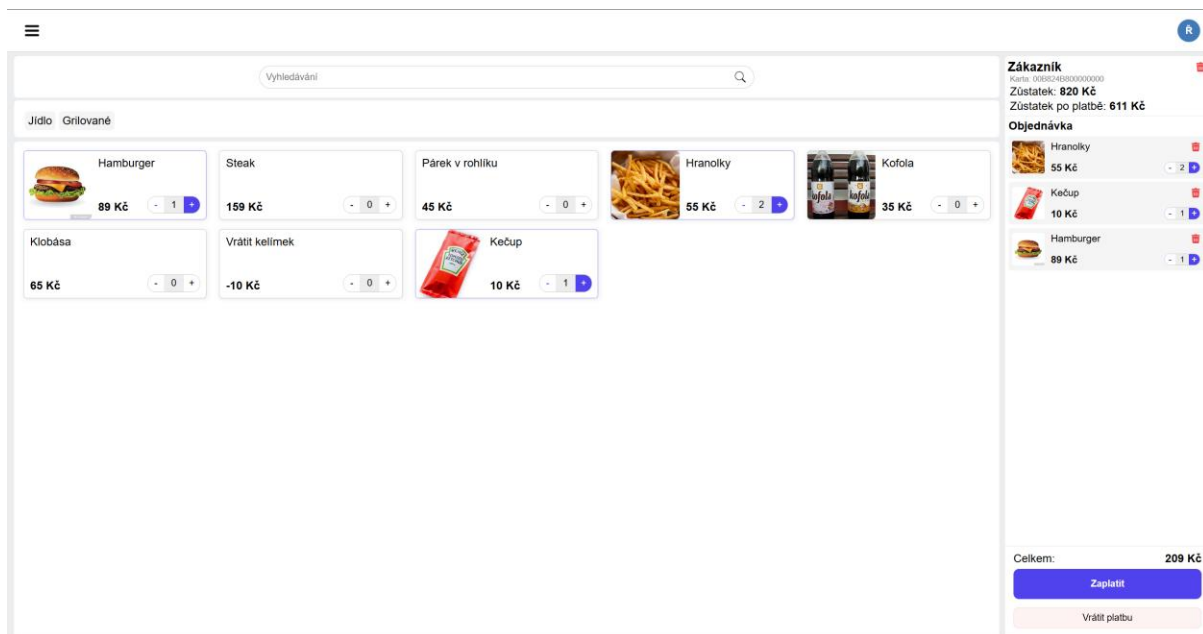
0

Zrušit

Vytvořit a přiřadit kartu

Obrázek 4 – Stránka cashier stánku
Zdroj: vlastní zpracování

- **Seller (prodávající) stánek** — zaměstnanec vidí seznam produktů (s obrázky a cenami), vybírá produkty do košíku, potvrzuje platbu. Může také provést refundaci poslední platby (viz Obrázek 5).

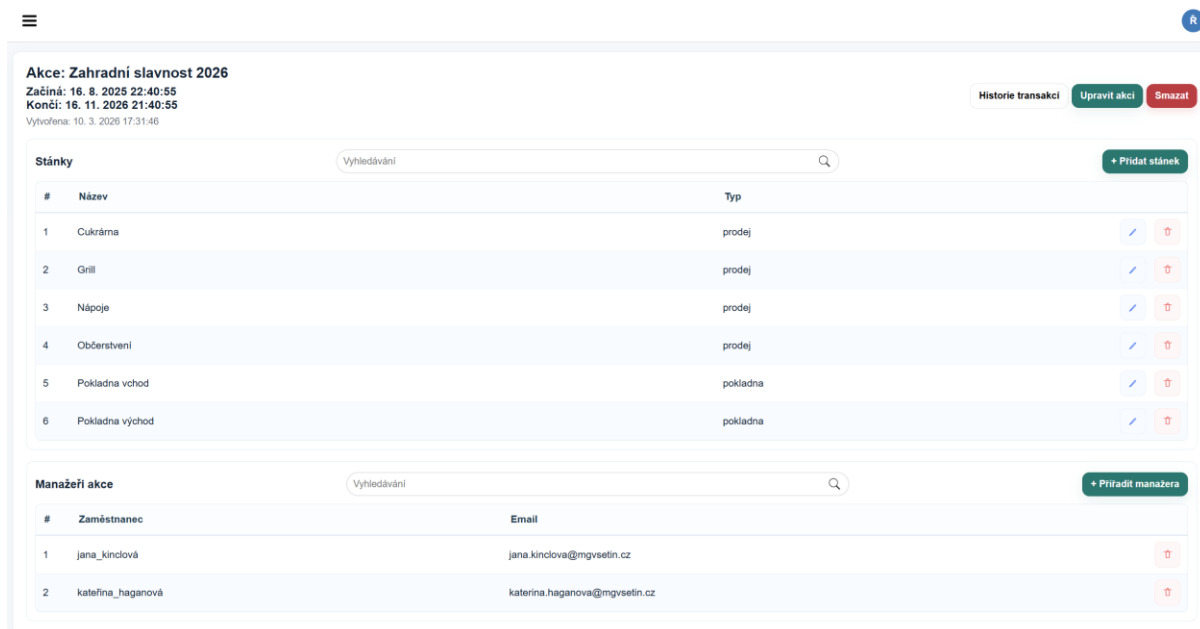


Obrázek 5 – Stránka seller stánku
Zdroj: vlastní zpracování

17.4 Správa akcí (event manager)

Obrázek 6 zobrazuje rozhraní pro správu akce, které umožňuje:

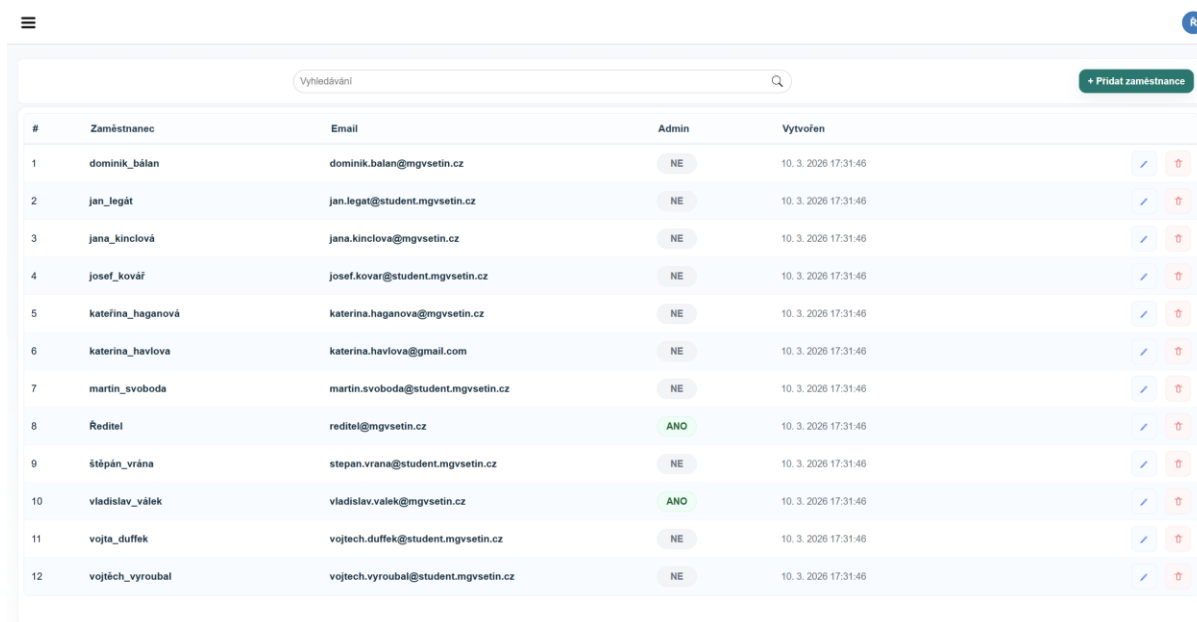
- Upravovat a smazat akci.
- Vytvářet, editovat a mazat stánky (cashier/seller).
- Spravovat produkty — přidávat, editovat, mazat, nahrávat obrázky, přiřazovat ke stánkům a kategoriím.
- Spravovat kategorie produktů a jejich přiřazení ke stánkům.
- Přiřazovat zaměstnance k rolím v rámci akce.
- Zobrazovat statistiky a historii transakcí.



Obrázek 6 – Stránka správy akcí
 Zdroj: vlastní zpracování

17.5 Správa zaměstnanců (admin)

Obrázek 7 zobrazuje stránku, na které může administrátor vytvářet nové zaměstnance, editovat jejich údaje (uživatelské jméno, email, heslo, stav admina) a mazat je. Aplikace zajistí, že musí být minimálně jeden administrátor.



Obrázek 7 – Stránka správy zaměstnanců
 Zdroj: vlastní zpracování

17.6 Správa smazaných záznamů

Jako praktický důsledek soft-delete vzoru obsahuje systém dedikované stránky pro zobrazení a obnovu smazaných záznamů. Administrátor může zobrazit smazané akce a administrátor, event manažer a zaměstnanci na cashier stánku mohou zobrazit smazané uživatele seřazené podle data smazání. U každého záznamu je možné jedním kliknutím provést obnovení (nastavení `deleted_at` zpět na NULL). Při obnově uživatele se automaticky obnoví i jeho peněženky, které byly smazány ve stejný okamžik. Při obnově akce se obnoví entity, které obsahovala při smazání, nikoli však vazby mezi nimi.

Obnovení může narazit na konflikty unikátnosti — například pokud byl po smazání vytvořen jiný uživatel se stejným e-mailem nebo kartu, kterou měl při smazání, teď používá někdo jiný. V takovém případě systém nabídne možnost „force“ obnovení, které automaticky vygeneruje unikátní alternativu (přidáním suffixu `_1`, `_2` atd. k identifikátoru, e-mailu, tagu peněženky), aby obnovení bylo vždy možné.

Obnovení bylo vytvořeno hlavně kvůli potřebě možnosti podívat se na historii plateb nebo vrácení zbývajících peněz zákazníkovi, proto je možné obnovit pouze uživatele a akce, u kterých se ani neobnoví vazby.

17.7 Typický scénář obsluhy

Pro lepší představu o každodenním používání systému jsou zde popsány dva typické scénáře (viz Obrázek 8):

17.7.1 Scénář pokladního (cashier):

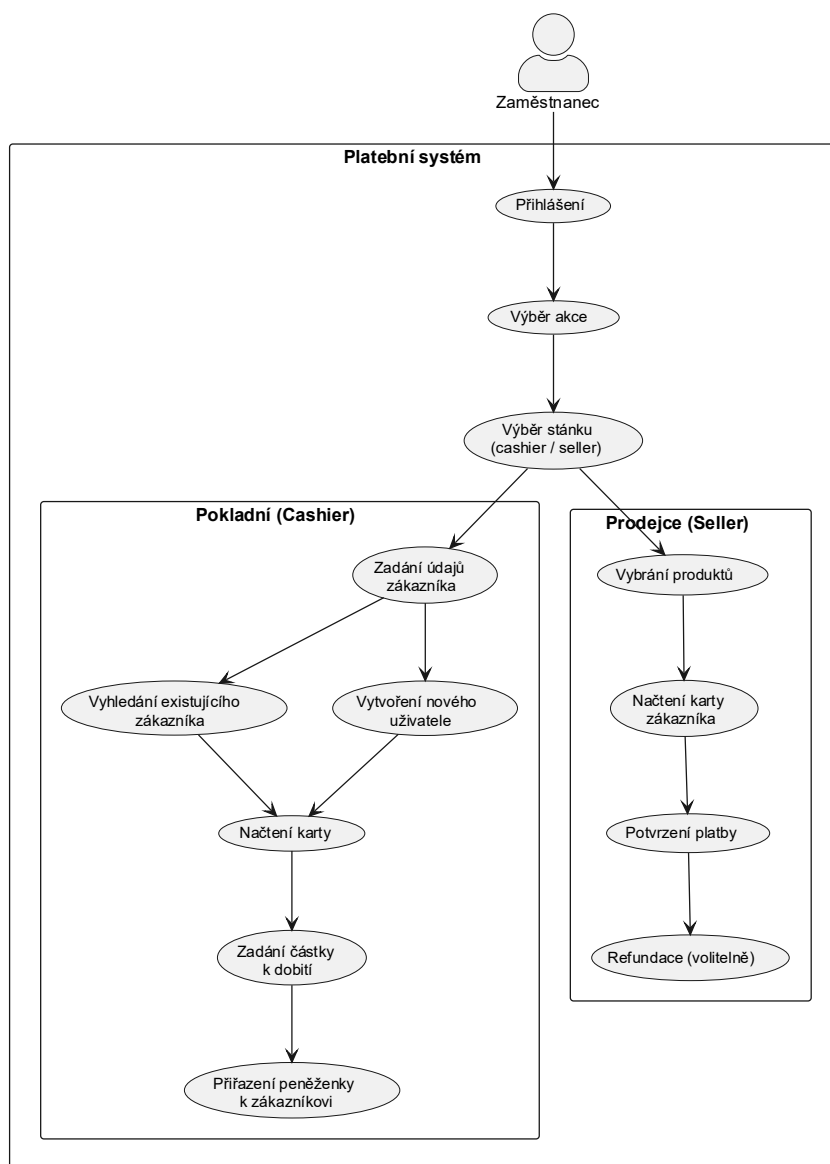
Pokladní se ráno přihlásí do systému, vybere dnešní akci (například „Školní jarmark 2026“) a svůj pokladní stánek. Přijde první návštěvník, který si chce nabít kartu. Pokladní zadá jméno, příjmení a volitelně email, telefonní číslo nebo jiný identifikátor návštěvníka. Pokud už existuje, tak ho pouze vybere, jinak vytvoří nového uživatele. Vytvoří mu peněženku přiložením návštěvníkovy RFID karty ke čtečce — systém automaticky načte tag ID karty a přiřadí ho k peněžence. Poté pokladní zadá částku (například 500 Kč), potvrdí dobítí a návštěvník může odejít a platit u prodejnách

stánků. Když se návštěvník vrátí a chce dobít další prostředky, stačí přiložit kartu — systém ho ihned identifikuje a pokladní pouze zadá novou částku. Na konci akce může návštěvník přijít pro výběr zbývajících prostředků a vrácení karty. Pokladní přiloží kartu ke čtečce a zvolí „vrátit peněženku“. Systém v rámci jedné databázové transakce vytvoří transakci typu balance-change se zápornou částkou odpovídající aktuálnímu zůstatku (čímž se zůstatek vynuluje) a poté peněženku soft-deletne. Pokladní vrátí návštěvníkovi hotovost odpovídající vybranému zůstatku a kartu přijme zpět.

17.7.2 Scénář prodejce (seller):

Prodejce si po přihlášení vybere akci a svůj prodejní stánek. Na obrazovce vidí vlevo seznam produktů rozdělený do kategorií (například „Jídlo“, „Nápoje“), vpravo košík s aktuální objednávkou. Když přijde zákazník a objedná si hamburger a limonádu, prodejce klikne na příslušné produkty — ty se přidají do košíku s celkovou cenou. Zákazník přiloží kartu ke čtečce a prodejce potvrdí platbu jedním kliknutím. Systém okamžitě odečte částku z peněženky a potvrdí platbu. Pokud zákazník zjistí, že dostal špatnou objednávku, nebo prodejce udělá jakoukoli chybu, prodejce může do 5 minut provést refundaci poslední platby. Celý proces jedné transakce tak trvá jen několik sekund.

Use Case Diagram -- Zaměstnanec (Cashier / Seller)



Obrázek 8 – Use case diagram cashier nebo seller zaměstnance
Zdroj: vlastní zpracování, planttext.com

18 Bezpečnostní implementace

18.1 Hashování hesel

Hesla jsou hashována algoritmem Argon2id s parametry `time_cost=3`, `memory_cost=65536` (64 MB), `parallelism=2`. Při každém přihlášení se kontroluje, zda parametry hashe odpovídají aktuální konfiguraci — pokud ne,

provede se automatický rehash. Tento mechanismus umožňuje transparentní zvýšení bezpečnostních parametrů v budoucnu bez nutnosti resetovat hesla.

18.2 Rate limiting

Omezení počtu požadavků chrání proti brute-force útokům a nadměrnému zatížení serveru. Globální limit je 200 požadavků za minutu. Endpoint pro přihlášení má přísnější limit 10 pokusů za 15 minut.

18.3 Validace nahrávaných souborů

Nahrávání obrázků produktů podléhá několika vrstvám validace:

- Maximální velikost souboru: 16 MB (`MAX_CONTENT_LENGTH`).
- Povolené MIME typy: `image/jpeg`, `image/png`, `image/webp`.
- Povolené přípony: `.jpeg`, `.jpg`, `.png`, `.webp`.
- Maximální rozlišení: 50 milionů pixelů.
- Obrázky jsou uloženy mimo adresář aplikace v konfigurovaném `UPLOAD_FOLDER`.
- Všechny kontroly jsou ověřeny i přes knihovnu Pillow, aby se zabránilo spoofingu⁶.
- Velikost obrázku je navíc ověřena při samotném ukládání, které se děje po částech a vždy se zkontroluje zda nebyla překročena hranice.

18.4 Ochrana cookies a sessions

Session cookie je zabezpečena nastavením:

- `HttpOnly = True` — cookie není přístupná z JavaScriptu, čímž se eliminuje riziko krádeže session ID prostřednictvím XSS útoku.
- `SameSite = Lax` — cookie se neodesílá při cross-site požadavcích (ochrana proti CSRF).

⁶ Spoofing v tomto kontextu označuje pokus o podvržení skutečného typu souboru (např. změnou přípony nebo MIME typu), aby neplatný či škodlivý soubor prošel kontrolou jako obrázek. Dodatečná validace pomocí knihovny Pillow ověřuje reálný obsah souboru, a tím takovým útokům předchází.

- `Secure = True` (v produkci) — cookie se odesílá pouze přes HTTPS.

18.5 Threat model — proti čemu se systém brání

Systém je navržen s ohledem na hrozby typické pro platební systémy provozované na akcích s fyzickým přístupem obsluhy k zařízením:

Hrozby, proti kterým se systém aktivně brání:

- **Neoprávněný přístup k systému** — řešeno autentizací (Argon2 hesla), hierarchií rolí a rate limitingem přihlašování.
- **Manipulace s finančními záznamy** — transakce jsou na úrovni databáze neměnné (triggery blokují UPDATE/DELETE). Soft-delete vzor zajišťuje, že žádná data nejsou fyzicky smazána. Finanční záznamy nemůže měnit ani administrátor.
- **Duplictní provedení platby** — mechanismus idempotence s SHA-256 fingerprintem zabraňuje dvojitému stržení prostředků.
- **SQL injection a XSS** — parametrizované dotazy na backendu a escapování uživatelských dat na frontendu (funkce `escapeHTML()`).
- **Neoprávněná finanční operace** — dvojí ověření oprávnění (aplikační vrstva + databázový trigger) zajišťuje, že ani chyba v aplikační logice neumožní neoprávněnou transakci.
- **Krádež session** — `HttpOnly` a `SameSite` cookies, volitelné vynucování IP adresy a `User-Agenta`.

18.6 Validace vstupních dat

Veškerá data přijatá od klienta procházejí validací na straně serveru předtím, než jsou zapsána do databáze. Validací funkce jsou soustředěny v modulu `utils/employees_users.py` (pro údaje osob) a v modulech `utils/products.py` a `utils/events.py` (pro produkty a akce). Každá validační funkce vrací dvojici (`is_valid`, `errors`), kde `errors` je seznam konkrétních chybových zpráv.

Validace uživatelského jména:

- Délka mezi 3 a 40 znaky.
- Začíná a končí alfanumerickým znakem (včetně diakritiky — rozsah Latin-1⁷ a Latin-Extended-A⁸).
- Povolené vnitřní znaky: písmena, číslice a znaky . _ -.
- Žádné po sobě jdoucí speciální znaky (například „..” nebo „_”).
- Volitelně lze zakázat čistě číselná jména a rezervované podřetězce.

Validace e-mailu:

- Využívá knihovnu email-validator, která ověřuje syntaxi podle RFC 5321/5322⁹.

Validace telefonního čísla:

- Využívá knihovnu phonenumbers (port Google libphonenumber¹⁰).
- Číslo je parsováno včetně kódu země, ověřeno pomocí is_possible_number() i is_valid_number().
- Platné číslo je uloženo ve formátu E.164¹¹ (například +420123456789) pro jednoznačnost.
- Na frontendu je pro zadávání telefonního čísla implementována vlastní komponenta s výběrem předvolby země. (viz Příloha 1)

Validace hesla:

- Minimální délka 8 znaků.
- Vyžadováno alespoň jedno velké písmeno, jedno malé písmeno, jedna číslice a jeden speciální znak.
- Zakázány mezery a tabulátory.
- Detekce příliš dlouhých sekvencí opakujících se znaků (6 a více).

⁷ **Latin-1 Supplement (U+00C0–U+00FF):** Zahrnuje diakritická písmena západoevropských jazyků (např. à, ä, ç, ñ, ö, ü); vyloučeny jsou nealfanumerické symboly ve stejném bloku (× U+00D7, ÷ U+00F7 a další)

⁸ **Latin Extended-A (U+0100–U+017F):** Pokrývá písmena středo- a východoevropských jazyků, včetně češtiny, slovenštiny, polštiny nebo chorvatštiny (např. č, š, ž, ř, ů, ł).

⁹ RFC 5321 definuje protokol SMTP a formát e-mailových adres v obálce zprávy. RFC 5322 specifikuje formát internetových zpráv včetně hlaviček. Obě normy společně definují, co je platná e-mailová adresa.

¹⁰ Knihovna libphonenumber je open-source projekt společnosti Google původně vyvinutý pro Android. Pythonový port zachovává aktuální metadata telefonních čísel pro všechny země světa.

¹¹ E.164 je doporučení ITU-T (Mezinárodní telekomunikační unie), které definuje maximálně 15místný formát mezinárodních telefonních čísel včetně kódu země.

- Kontrola, zda heslo neobsahuje uživatelské jméno nebo lokální část e-mailu.

Validace jmen (křestní jméno a příjmení):

- Délka mezi 1 a 100 znaky.
- Povolené znaky: písmena (včetně diakritiky), číslice a znaky . _ -.
- Databázový trigger automaticky převádí první písmeno na velké.

Validace produktů a akcí:

- Název produktu/kategorie/akce/stánku: neprázdný řetězec s omezenou délkou a povolenými znaky.
- Cena produktu: celé číslo v definovaném rozsahu (záporná cena je povolena — například pro vratné kelímky).
- UUID identifikátory: validovány parsováním do třídy UUID, neplatný formát vrací chybu 400.

Validace finančních hodnot:

- Částky dobítí a zůstatky musí být celá čísla v rozsahu -1 000 000 až 1 000 000 Kč.
- Při vytváření peněženky se ověřuje shoda mezi požadovanou změnou zůstatku a novým zůstatkem (ochrana proti manipulaci na straně klienta).

Všechny validační chyby jsou klientovi vráceny ve formátu JSON s klíčem error (a volitelně detail pro upřesnění, kterého pole se chyba týká) a odpovídajícím HTTP kódem (400 pro neplatný vstup, 409 pro konflikt unikátnosti). Klient podle chyb zobrazí uživateli odpovídající chybnou hlášku.

18.6.1 Co je mimo scope systému

- **Fyzická bezpečnost čteček a zařízení** — systém neposkytuje ochranu proti fyzické manipulaci s hardwarem (například výměna čtečky za podvržené zařízení). Předpokládá se, že organizátor akce zajišťuje fyzickou bezpečnost stánků.
- **Šifrování komunikace karta–čtečka** — RFID/NFC tag ID je přenášeno v otevřené podobě. Systém se spoléhá na krátký dosah NFC a na to, že tag ID samotné bez přístupu do systému nemá hodnotu.

- **DDoS útoky na infrastrukturu** — základní rate limiting chrání proti jednoduchým útokům, ale ochrana proti distribuovaným útokům závisí na infrastruktuře (firewall, reverzní proxy).

19 Plánované úlohy na pozadí a logování

19.1 Plánované úlohy

Aplikace využívá APScheduler pro periodické úlohy na pozadí:

- **Čištění sessions** — každých 60 minut se smažou sessions, které jsou neaktivní déle než nakonfigurovaný limit (výchozí: 7 dní).
- **Čištění nepoužívaných obrázků** — každé 3 hodiny se identifikují obrázky, na které neodkazuje žádný produkt, a odstraní se z databáze i disku.
- **Čištění sirotčích obrázků na disku** — každých 12 hodin se zkontroluje, zda na disku nejsou soubory, které nemají odpovídající záznam v databázi.

Pro prostředí s více WSGI workery (například Gunicorn) zajišťuje filelock¹², že scheduler běží pouze v jednom z workerů, čímž se zabráňuje duplicitnímu provádění úloh.

19.1.1 Zálohování databáze

Volitelná periodická úloha, která vytváří zálohu databáze pomocí pg_dump v komprimovaném custom formátu (-Fc). Interval se nastavuje konfigurační hodnotou `SCHEDULER_BACKUP_MINUTES` (0 = vypnuto). Zálohy se ukládají do adresáře `BACKUP_DIR` (výchozí: instance/backups/) s názvem ve formátu `backup_YYYY-MM-DD_HHMMSS.dump` a automatická rotace odstraňuje nejstarší zálohy nad limit `BACKUP_MAX_COUNT`.

¹² Filelock je Pythonová knihovna implementující souborové zámky nezávislé na platformě. V tomto kontextu zajišťuje, že plánovací úlohy (scheduler) běží pouze v jednom procesu.

Kromě automatického zálohování na pozadí je možné zálohu vytvořit i ručně pomocí CLI příkazu:

```
flask --app cashier_app backup-db
```

Příkaz vytvoří zálohu stejným způsobem jako plánovaná úloha a vypíše cestu k vytvořenému souboru.

Pro obnovu databáze ze zálohy slouží CLI příkaz:

```
flask --app cashier_app restore-db [cesta_k_souboru]
```

Pokud není cesta k souboru zadána, automaticky se použije nejnovější záloha z adresáře `BACKUP_DIR`. Obnova se provádí pomocí `pg_restore` s přepínači `--clean --if-exists --single-transaction --exit-on-error`, což zajišťuje, že obnova proběhne atomicky — pokud jakýkoliv krok selže, celá transakce se vrátí zpět (rollback) a databáze zůstane v původním stavu.

19.2 Logování

Pro finanční systém je logování důležitou součástí provozu — umožňuje zpětně dohledat příčinu problémů, sledovat neočekávané chování a ověřovat správný běh systému. Aplikace využívá standardní Python logging modul integrovaný do Flasku (`current_app.logger`):

- **Chyby (exception)** — neočekávané výjimky při operacích se zaměstnanci, peněženkami, událostmi, produkty a dalšími entitami jsou logovány s kompletním tracebackem pomocí `current_app.logger.exception()`.
- **Varování (warning)** — konflikty při undo/redo operacích (například pokus o obnovení entity smazané jiným uživatelem) jsou logovány jako varování, aby bylo možné zpětně zjistit, proč operace neskončila standardním způsobem.
- **Informační záznamy (info)** — plánované úlohy na pozadí logují svůj průběh: počet vyčištěných sessions, dokončení čištění obrázků a další provozní informace.

- **Chyby scheduleru** — všechny plánované úlohy jsou obaleny try-except bloky a případné výjimky jsou zaznamenány pomocí `logger.error()`, aby selhání jedné úlohy nenarušilo běh ostatních.

V produkčním nasazení lze logování nakonfigurovat na výstup do souboru nebo externího systému.

19.2.1 Logování transakcí

Samotné finanční transakce nejsou logovány do log souboru — jejich kompletní a neměnná historie je uložena přímo v databázi (tabulka `transactions`), což slouží jako primární auditní záznam.

20 Frontend a uživatelské rozhraní

20.1 Architektura frontendu

Frontend je postaven na čistém JavaScriptu bez použití frameworků (React, Vue, Angular). Kód je organizován do ES modulů, kde každý modul odpovídá za jednu funkční oblast (produkty, uživatelé, peněženky, objednávky, karty atd.). HTML šablony jsou generovány na serveru pomocí Jinja2 a poskytují základní strukturu stránky, zatímco veškerý dynamický obsah je načítán a aktualizován pomocí JavaScriptu.

Komunikace se serverem probíhá výhradně přes `fetch()` API. Dynamický HTML obsah se vytváří pomocí template literálů a vkládá do DOM. Všechna uživatelská data jsou před vložením do HTML escapována funkcí `escapeHTML()` pro ochranu proti XSS útokům.

20.2 Event delegation

Pro efektivní zpracování událostí na dynamicky generovaném obsahu se používá vzor event delegation — posluchače událostí jsou registrovány na elementu `document` a pomocí kontroly `event.target` se identifikuje konkrétní element, na

kterém událost nastala. Tento přístup eliminuje nutnost přiřazovat posluchače jednotlivým elementům po každém překreslení obsahu.

20.3 Vyhledávání a řazení tabulek

Všechny tabulky v aplikaci disponují klientským vyhledáváním a řazením. Veškerá logika filtrování i řazení probíhá na straně frontendu v JavaScriptu — backend poskytuje data bez parametrů pro vyhledávání či řazení.

Tento přístup byl zvolen záměrně z několika důvodů. Filtrování a řazení na klientovi je pro uživatele výrazně rychlejší, protože operace probíhají okamžitě nad již načtenými daty bez nutnosti odesílat nový HTTP požadavek na server a čekat na odpověď. Zároveň se tím snižuje zátěž na server a databázi — při každém stisku klávesy ve vyhledávacím poli by server musel opakovaně zpracovávat SQL dotazy, zatímco na klientovi jde pouze o filtrování pole objektů v paměti. Předpokladem tohoto řešení je, že se při načtení stránky stáhnou všechna data dané tabulky najednou. To je pro tento systém přijatelné, protože očekávané objemy dat (desítky až stovky zaměstnanců, akcí,...) nepředstavují problém pro přenos ani pro paměť prohlížeče.

20.3.1 Vyhledávání

Každá stránka s tabulkou obsahuje vyhledávací pole (HTML input s třídou `.search-bar`), které při každém stisku klávesy v tomto poli (událost `input`) okamžitě přefiltruje zobrazené záznamy. Filtrování probíhá voláním funkce typu `isSearchedFor()`, která přijímá objekt, o kterém rozhoduje zda je vyhledávaný, a aktuální text z vyhledávacího pole.

Vyhledávání podporuje dva režimy:

- **Volný text** — zadaný řetězec se hledá napříč všemi zobrazenými sloupci záznamu. Například zadáním „jan“ se zobrazí záznamy obsahující „jan“ v jakémkoli poli (jméno, e-mail, identifikátor apod.).
- **Vyhledávání podle konkrétního pole** — syntaxe `klíč=hodnota` umožňuje omezit hledání na jeden sloupec. Například `username=admin` vyhledá pouze

v poli uživatelského jména. Klíče podporují české i anglické varianty (např. jméno=, email=, vytvořen=, telefon= a další).

Více hledaných výrazů oddělených mezerou funguje jako konjunkce — záznam musí odpovídat všem zadaným výrazům současně. Vyhledávání je case-insensitive.

Příklad funkce pro filtrování zaměstnanců (employees_manager.js):

```
function isSearchedFor(employee, searchQuery) {
  if (!searchQuery) return true;
  const searchQueries = searchQuery.toLowerCase().trim().split(/\s+/);
  // ...pro každý výraz kontrola, zda záznam odpovídá
  for (const query of searchQueries) {
    if (!query.includes('=')) {
      if (!employeeInfo.includes(query)) return false;
    } else {
      const searchKeyword = query.split('=')[0];
      const search = query.split('=')[1];
      // porovnání s konkrétním polem dle klíče
    }
  }
  return true;
}
```

20.3.2 Řazení

Záhlaví tabulek (elementy <th>) slouží zároveň jako tlačítka pro řazení — kliknutím na název sloupce se aktivuje třífázové přepínání:

1. První kliknutí — vzestupné řazení (ascending), zobrazí se šipka dolů (↓).
2. Druhé kliknutí — sestupné řazení (descending), zobrazí se šipka nahoru (↑).
3. Třetí kliknutí — řazení se deaktivuje, šipka zmizí a data se zobrazí v původním pořadí.

Aktuální stav řazení je uložen v objektu orderBy s vlastnostmi key (název sloupce) a ascending (boolean). Funkce toggleOrder() přepíná mezi těmito třemi stavy:

```
const orderBy = { key: '', ascending: true };

function toggleOrder(key) {
  if (orderBy.key !== key) {
    orderBy.key = key;
    orderBy.ascending = true;
  } else if (orderBy.ascending) {
    orderBy.ascending = false;
  } else {
    orderBy.key = '';
    orderBy.ascending = true;
  }
}
```

Řazení je typově citlivé — textové sloupce (jméno, e-mail) se řadí pomocí `localeCompare()` s převedením na malá písmena, datumové sloupce (vytvořen, začátek, konec) se řadí porovnáním objektů `Date`. Vizuální indikátor řazení (šipka) se dynamicky přidává do záhlaví aktivního sloupce jako element `<span class="order-by-arrow">`.

Po každé změně filtru nebo řazení se volá `loadPage({table: true})`, která překreslí tabulku s aktuálně platnými parametry vyhledávání i řazení.

21 Testování

Projekt obsahuje rozsáhlou sadu automatizovaných testů implementovaných pomocí frameworku `pytest`. Testy pokrývají jak izolovanou logiku jednotlivých komponent, tak integrační scénáře zahrnující skutečnou databázi. Coverage¹³ testů je 86 %.

¹³ Coverage testů je procento zdrojového kódu, které bylo při spuštění testů vykonáno; obvykle se počítá podle řádků kódu. Neznamená to automaticky kvalitu testů, ale ukazuje, jak velká část řádků byla testy alespoň jednou spuštěná.

21.1 Unit testy a integrační testy

Testy v projektu lze rozdělit do dvou kategorií:

- **Unit testy** — testují izolovanou logiku bez závislosti na databázi. Například testy v `test_undo_redo.py` ověřují správné řazení změn pro operace undo a redo (rodičovské záznamy se obnovují před dětskými, aby nedošlo k porušení referenční integrity), detekci typu změny (insert/update/delete) a filtrování neúčinných změn.
- **Integrační testy** — většina testů v projektu pracuje s reálnou PostgreSQL databází. Sdílený fixture `db_pool` v souboru `confest.py` vytváří connection pool k testovací databázi, inicializuje schéma a po každém testu provede rollback, aby byl zajištěn čistý stav. Testy tak ověřují celý řetězec od HTTP požadavku přes aplikační logiku, SQL dotazy a databázové triggeru až po odpověď.

21.2 Kritické scénáře — platby a refundace

Testy finančních operací patří k nejdůležitějším, protože chyba v platební logice má přímý finanční dopad:

- **Průběh platby** — testy ověřují kompletní průběh platby včetně správného výpočtu `balance_before/balance_after`, odečtení z peněženky a vytvoření záznamu transakce. Testují se jak úspěšné platby, tak odmítnutí při nedostatečném zůstatku (`InsufficientBalanceError`).
- **Idempotence** — testy ověřují, že opakovaný požadavek se stejným idempotentním klíčem nevytvoří duplicitní transakci, a že požadavek se stejným klíčem, ale jinými parametry, vrátí chybu 409 (`IdempotencyKeyDataConflict`).
- **Refundace** — testuje se, že refundace vytvoří novou transakci s kladnou částkou, správně odkáže na původní transakci a že nelze refundovat již refundovanou transakci.

- **Souběžné transakce** — testy simulují souběžný přístup ke stejné peněžence z více vláken a ověřují, že řádkové zámky (FOR UPDATE) zajistí konzistentní stav i při paralelním zpracování.

21.3 Testy undo/redo a paste

- **Undo/redo** — testy ověřují správné pořadí aplikace změn (při undo se smazané záznamy obnovují před dětskými, při redo se vkládají v originálním pořadí), detekci typu operace a filtrování změn, které by neměly žádný efekt.
- **Paste/klonování** — testy pokrývají klonování akcí, zaměstnanců a jejich přiřazení, generování unikátních názvů (sufix `_copy`, `_copy2` atd.) a ověření autorizace (pouze admin může klonovat zaměstnance a akce).

21.4 Další testované oblasti

- **Autentizace** — přihlášení, odhlášení, ověřování hesel s rehashováním, rate limiting.
- **CRUD operace** — vytváření, editace, mazání a obnovování záznamů, validace vstupů, kontrola oprávnění.
- **Databázové trigger** — testy ověřují, že trigger správně blokuje fyzické smazání, chrání neměnné sloupce a normalizují data.
- **Validace vstupů** — testy pokrývají chybné formáty (neplatné UUID, záporné ceny, příliš dlouhé řetězce) a ověřují, že server vrací odpovídající HTTP kódy (400, 401, 403, 409).
- **Zálohování a obnova databáze** — unit testy ověřují správné volání `pg_dump/pg_restore` s očekávanými argumenty, rotaci starých záloh, vyhledání nejnovější zálohy a chování CLI příkazů. Integrační testy proti skutečné databázi ověřují, že záloha a následná obnova zachová data do původního stavu a že obnova z poškozeného souboru selže bez narušení databáze (transakční rollback).

Testy se spouštějí příkazem `pytest` a pro jejich běh je vyžadována běžící instance PostgreSQL s testovací databází.

ZÁVĚR

Hlavním cílem této práce bylo navrhnout a implementovat funkční, bezpečný a snadno nasaditelný bezhotovostní platební systém pro hromadné akce. Tento cíl byl splněn — výsledkem je webová aplikace, která umožňuje kompletní správu bezhotovostních plateb prostřednictvím RFID/NFC karet, od nabití kreditu přes platby u prodejních stánků až po statistiky a historii transakcí.

Splnění dílčích cílů

1. **Správa více souběžných akcí** — cíl byl splněn. Systém plně podporuje více akcí probíhajících současně, každá s vlastní sadou stánků, produktů, kategorií a zaměstnanců. Čtyřúrovňová hierarchie rolí (admin, event manager, cashier, seller) zajišťuje, že každý zaměstnanec vidí a může spravovat pouze akce a stánky, ke kterým má oprávnění.
2. **Integrita a bezpečnost finančních dat** — cíl byl splněn. Kombinace ACID transakcí PostgreSQL, databázových triggerů blokujících modifikaci transakcí, řádkových zámků na peněženkách a mechanismu idempotence zajišťuje konzistenci finančních dat. Dvojí vrstva ověřování oprávnění (aplikační logika + databázový trigger) minimalizuje riziko neoprávněných operací. Projekt obsahuje automatizované testy, které ověřují správnost finančních operací včetně souběžného přístupu.
3. **Statistiky a přehledy** — cíl byl splněn. Organizátoři mají k dispozici detailní statistiky na úrovni celé akce, jednotlivých stánků i produktů, včetně časových průběhů a žebříčků. Kompletní historie transakcí je přístupná jak pro jednotlivé uživatele, tak pro celou akci.

Odchyłky od zadání

Offline status — systém vyžaduje připojení k databázovému serveru. Plně offline režim by vyžadoval lokální databázi s pozdější synchronizací, což by výrazně zvýšilo

složitost projektu bez přiměřeného přínosu pro typické nasazení, kde je k dispozici lokální síť.

Silné stránky

- **Robustní zabezpečení finančních dat** — přesunutí klíčové business logiky do databázových triggerů zajišťuje integritu dat nezávisle na aplikační vrstvě. Transakce jsou na úrovni databáze neměnné — ani administrátor je nemůže modifikovat nebo smazat.
- **Rozsáhlé automatizované testování** — 613 testů ve 28 souborech pokrývá validaci vstupů, finanční operace, souběžný přístup, databázové trigger, autorizaci, zálohování a další oblasti. Testy pracují s reálnou PostgreSQL databází, nikoliv s mocky.
- **Nezávislost na externích službách** — aplikace nevyžaduje žádné třetí strany pro svůj provoz. Stačí Python, PostgreSQL a webový prohlížeč. Pro čtení karet není potřeba instalovat žádný software ani ovladače díky Web Serial API.

Slabé stránky a omezení

- **Závislost na Chromium prohlížečích** — Web Serial API není podporováno ve Firefoxu ani Safari, což omezuje výběr prohlížeče pro čtení karet. Pro správu akcí a statistiky je však možné použít libovolný prohlížeč.
- **Absence offline režimu** — při výpadku připojení k databázi není možné provádět žádné transakce. V prostředí lokální sítě na akci je toto riziko nízké, ale pro vzdálené nasazení by to bylo omezující.
- **Načítání všech dat najednou** — frontend při načtení stránky stahuje kompletní obsah tabulky (zaměstnanci, akce, smazané záznamy) a filtrování a řazení provádí lokálně v prohlížeči. Při očekávaných objemech dat (desítky až stovky záznamů) to nepředstavuje problém, ale při výrazně větším množství záznamů (tisíce a více) by bylo vhodné přejít na stránkování a serverové filtrování.

- **Chybějící export dat** — statistiky a historie transakcí jsou dostupné pouze v rámci webového rozhraní. Chybí možnost exportu do formátů CSV nebo PDF pro další zpracování.

Možnosti budoucího rozšíření

- **Export statistik** — implementace exportu do CSV a PDF by organizátorům umožnila zpracovávat data v externích nástrojích. Díky modulární struktuře API by stačilo přidat nové endpointy, které vrátí data v požadovaném formátu.
- **Podpora mobilních zařízení s NFC** — moderní telefony s Android podporují čtení NFC tagů prostřednictvím Web NFC API, což by umožnilo použít telefon jako čtečku karet místo dedikovaného USB zařízení.
- **Online dobíjení** — integrace platební brány (například Stripe¹⁴) by umožnila návštěvníkům dobíjet kredit předem z vlastního zařízení, čímž by se zkrátily fronty u pokladních stánků.
- **Vícejazyčná podpora** — rozhraní je v současnosti česko-anglické. Přidání plné vícejazyčné podpory by rozšířilo použitelnost systému pro mezinárodní akce.

¹⁴ Stripe je americká technologická společnost poskytující platební infrastrukturu pro internetové podnikání. Alternativami jsou například GoPay nebo Comgate pro český trh.

SEZNAM POUŽITÉ LITERATURY

1. ANTHROPIC. Claude Opus 4.6 [velký jazykový model]. 2026. Dostupné z: <https://claude.ai/>
2. CODD, Edgar F. A Relational Model of Data for Large Shared Data Banks. **Communications of the ACM** [online]. 1970, roč. 13, č. 6, s. 377–387 [cit. 2026-03-28]. Dostupné z: <https://dl.acm.org/doi/10.1145/362384.362685>
3. MOZILLA. Web Serial API – Web APIs. **MDN Web Docs** [online]. ©2024 [cit. 2026-03-28]. Dostupné z: https://developer.mozilla.org/en-US/docs/Web/API/Web_Serial_API
4. NFC FORUM. What is NFC? **NFC Forum** [online]. ©2024 [cit. 2026-03-28]. Dostupné z: <https://nfc-forum.org/learn/what-is-nfc/>
5. OWASP FOUNDATION. Password Storage Cheat Sheet. **OWASP Cheat Sheet Series** [online]. ©2024 [cit. 2026-03-28]. Dostupné z: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
6. PALLETS PROJECTS. Flask Documentation [online]. ©2024 [cit. 2026-03-28]. Dostupné z: <https://flask.palletsprojects.com/en/stable/>
7. PASSWORD HASHING COMPETITION [online]. [cit. 2026-03-28]. Dostupné z: <https://www.password-hashing.net/>
8. POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 17 Documentation [online]. ©2024 [cit. 2026-03-28]. Dostupné z: <https://www.postgresql.org/docs/current/>

SEZNAM PŘÍLOH

Příloha 1 – Seznam pro vybírání předčísí	77
Příloha 2 – Platební operace – flowchart.....	78

Příloha 1 – Seznam pro vybírání předčísí

Telefonní číslo

|

775937282

NEDÁVNO POUŽITÉ

CZ

Česká repu...
Česká republ...
+420

VŠECHNY ZEMĚ

Žádné

Žádné předčísí

US

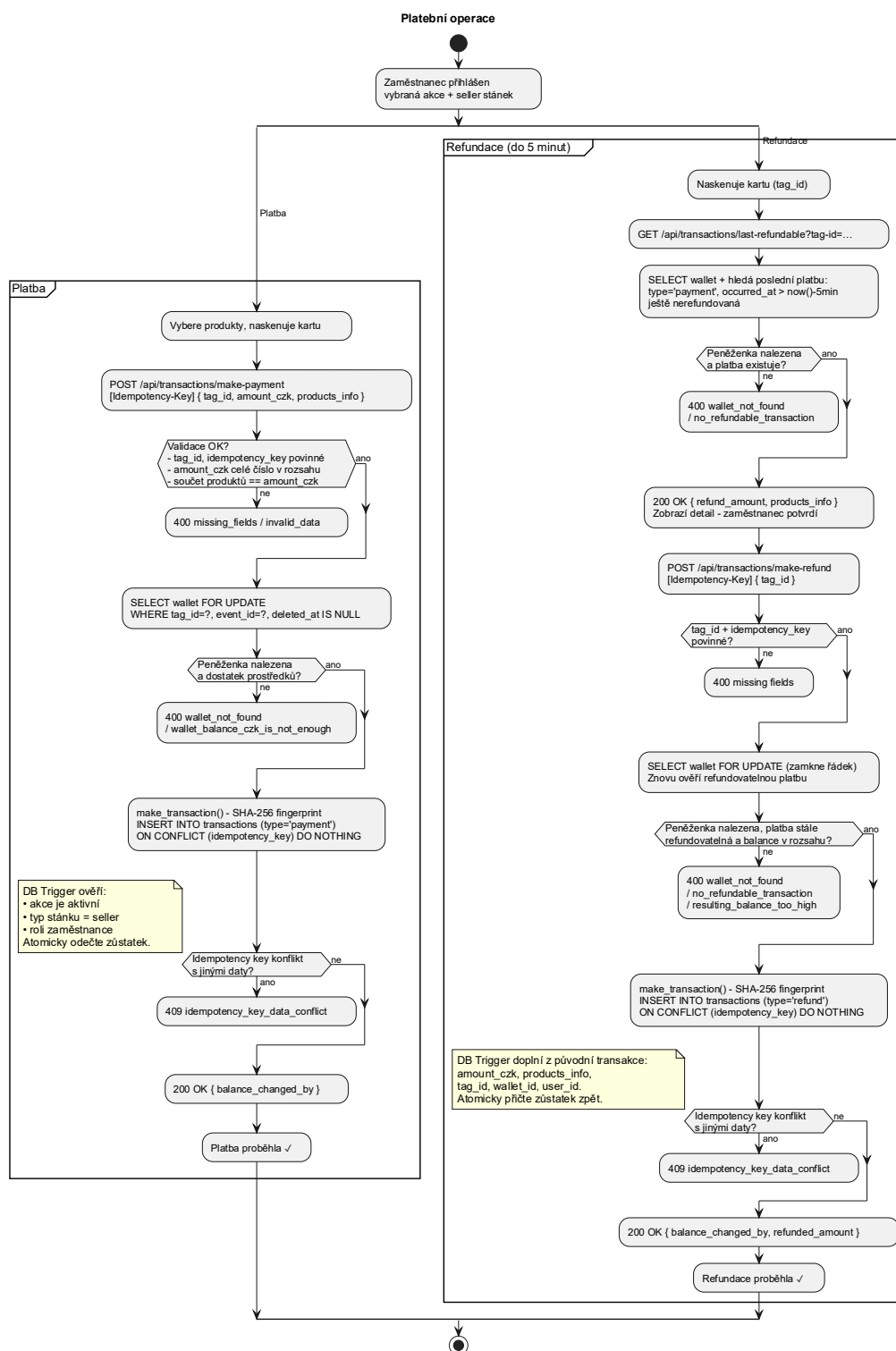
USA / Kanada
USA / Canada
+1

RU

Rusko
Россия
+7

Zdroj: vlastní zpracování

Příloha 2 – Platební operace – flowchart



Zdroj: vlastní zpracování, planttext.com